



iimas

Proyecto 1 OLTP : SPA

Bases de Datos Estructurados

Dra. Maria del Pilar Angeles

Equipo 2:

**Chuquiure Gil Ignacio
Pérez Martínez Ángel Noel
Rivera Hernández Milena Fernanda**

18 de septiembre del 2025

Hoja de requerimientos.

SPA

Se desea crear un Sistema de Base de Datos que contenga información sobre los clientes de una cadena de Spas muy importante.

El sistema deberá proporcionar información acerca de los clientes, empleados, instalaciones y servicios que ofrecen en dicho establecimiento.

Por cada cliente se desea conocer el paquete contratado, fecha en la que se realizó la contratación y el método de pago usado.

Por cada empleado se desea conocer el número de empleado, nombre, categoría en la que trabaja (gerente, recepción, masajista, etc.) y la instalación en la que se encuentra laborando.

Por cada instalación conocer el número de camas solares en ella, el número de focos que usa cada cama y las sesiones que se llevan a cabo en ella por día.

Existen paquetes disponibles para los clientes en cada instalación.

El sistema deberá contestar las siguientes preguntas:

- i) Dado el nombre de un empleado, conocer las sesiones que ha aplicado y los clientes que atendió en cada sesión.
- ii) Dado el nombre de una cama solar, conocer el nombre de los empleados que la usan con frecuencia y el número de sesiones que han realizado en ella.
- iii) Dado el nombre de un paquete, conocer el nombre de aquellos clientes que lo han contratado y el método de pago que han usado.

FORMATO DE ENTREGA DE PROYECTO:

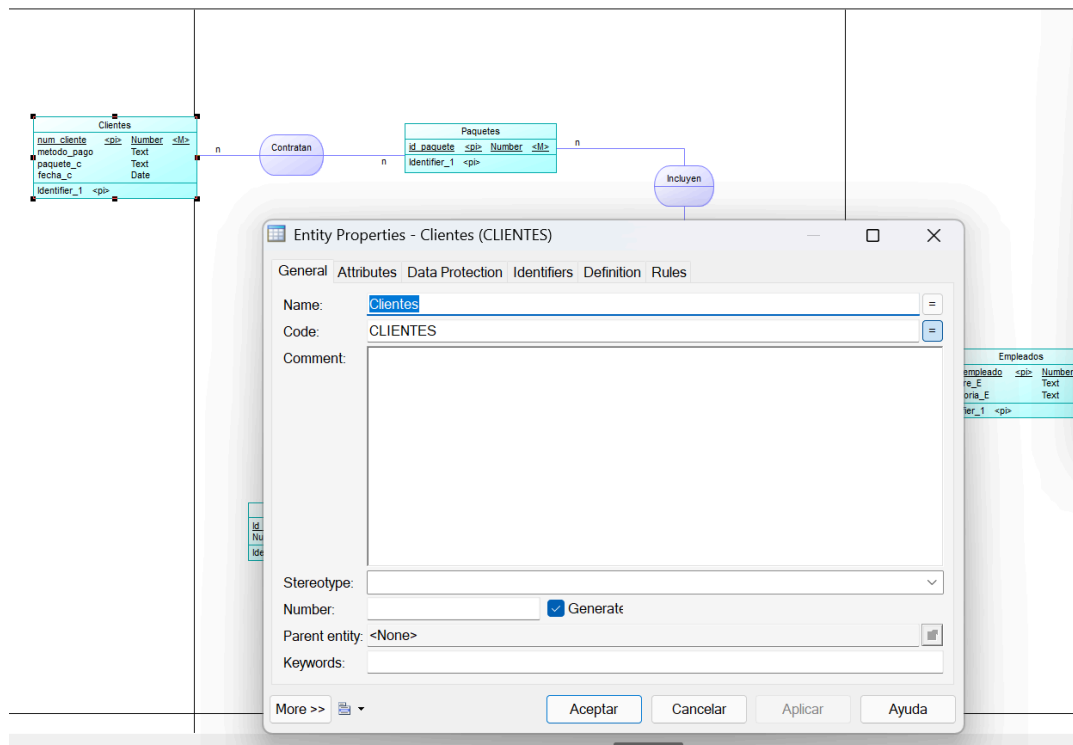
- Hoja de requerimientos.
- Diagrama Entidad - Relación realizado con PowerDesigner
- Modelo Relacional normalizado realizado con PowerDesigner
- Listados de programas en SQL que contengan lo siguiente:
 - Creación de base de datos con el nombre que se indica en el encabezado.
 - Creación de tablas.
 - Creación de llaves foráneas y primarias según corresponda con el modelo lógico.
 - Creación de reglas, constraints o defaults que permitan integridad de dominio y/o reglas de negocio.
 - Alta de cuando menos 10 registros a las tablas catálogo de la BD y los registros correspondientes en las tablas relación para denotar las asociaciones 1:1; 1:M; M:N. Donde M y N son cuando menos 3.
 - Creación de índices en campos líderes de consulta común.
 - Procedimiento almacenado sp_inserta que, a través de parámetros permita inserción de registros de alguna tabla catálogo.
 - Creación de consultas, que satisfagan la hoja de requerimientos.
 - Procedimiento almacenado sp_consultas que acepte parámetros y ejecute 5 consultas, si los requerimientos contienen menos crear dos nuevas. La salida del procedimiento (es decir, el resultado de las consultas) debe guardarlo en un archivo
 - Procedimiento almacenado sp_borra que permita eliminación de registros de alguna tabla catálogo.
 - Comandos necesarios para el control de la integridad de la información.

El proyecto debe entregarse en papel y en formato electrónico por correo electrónico a la misma dirección que las Prácticas. El documento en pdf debe contener los elementos que se especifican en el formato de entrega que se indica en la hoja de requerimientos, en el mismo orden. Al igual que las prácticas, código en formato txt y pantallas con la ejecución correcta de la creación, inserción de las tablas, así como la ejecución de altas, bajas y cambios en los datos que reflejen el aseguramiento de la integridad referencial (si el dato no es correcto, no permite, si lo es si permite), consultas que se piden, así como la interpretación de los resultados de los análisis. ES MUY IMPORTANTE QUE SE PROPORCIONE EL CÓDIGO PARA BORRAR LAS BASES DE DATOS, ASÍ COMO LOS CÓDIGOS PYTHON O RAPIDMINER USADOS.

Recopiló: Dra. María del Pilar Angeles

Desarrollo del Diagrama - Entidad Relación

- Creación de entidades



- Creación de relaciones

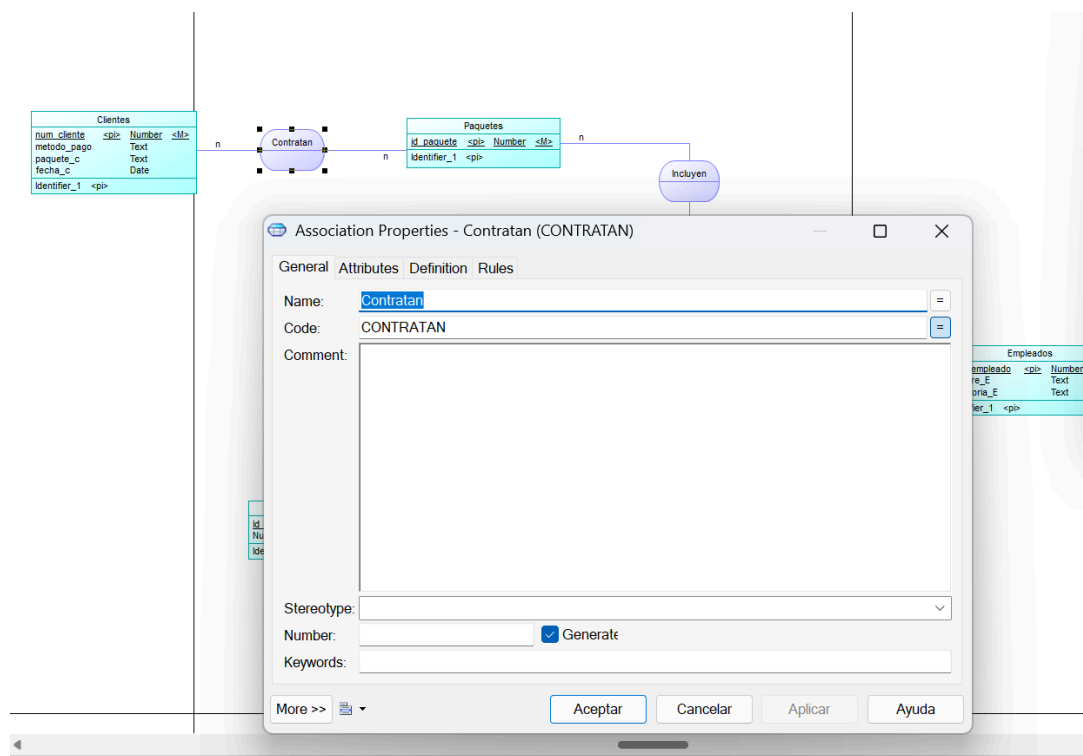
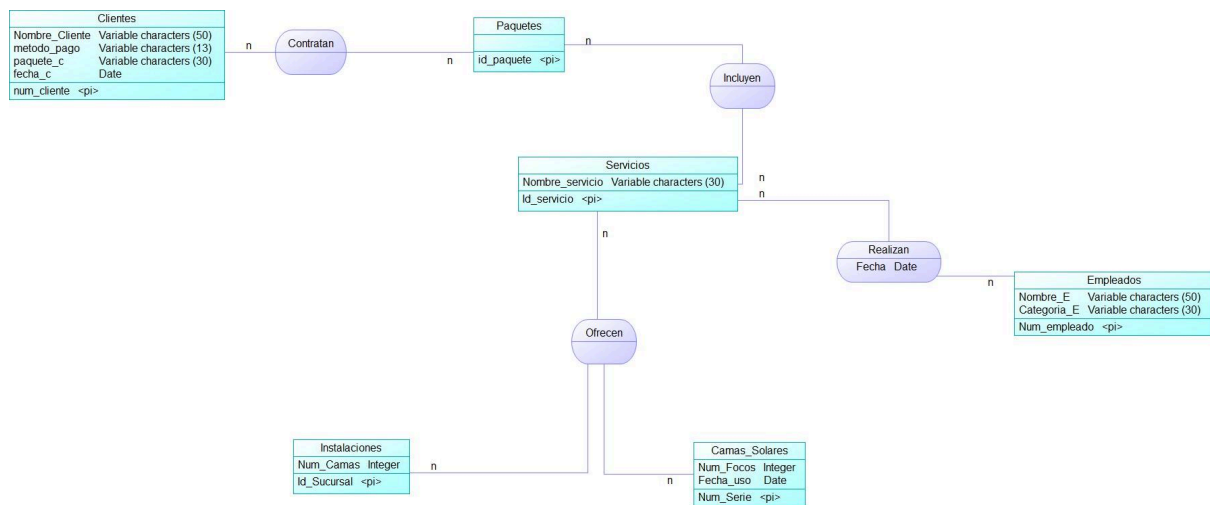


Diagrama Entidad-Relación



Normalización.

Antes de normalizar tenemos:

Clientes(num_cliente CHAR(8), Nombre_cliente VARCHAR(50),
metodo_pago VARCHAR(14), paquete_c VARCHAR(30), fecha_c DATE)

Paquetes(id_paquete CHAR(8))

Servicios(id_servicio CHAR(8), nombre_servicio VARCHAR(30))

Empleados(Num_empleado CHAR(8), Nombre_E VARCHAR(50),
Categoria_E VARCHAR(30))

Camas_Solares(Num_Serie CHAR(8), Num_Focos INTEGER, Fecha_uso
DATE)

Instalaciones(Id_Sucursal CHAR(8), Num_Camas INTEGER)

Contratan(num_clientes CHAR(8), id_paquete CHAR(8))

Incluyen(id_paquete CHAR(8), id_servicio CHAR(8))

Realizan(id_servicio CHAR(8), Num_empleado CHAR(8), Fecha DATE)

Ofrecen(id_servicio CHAR(8), Id_Secursal CHAR(8), Num_Serie VARCHAR(8))

Nota: como no podemos poner 3 líneas para subrayar la llave primaria compuesta, se verá así: llaves y las foráneas como: foránea

1FN: Observamos que los dos datos multivariados que encontramos están en el conjunto de entidades Empleados con 'Nombre_E' y Clientes con 'Nombre_c', ya que se puede dividir en Nombres_E/c, Apellido_paterno_E/c, Apellido_materno_E/c. NOTA: Por practicidad cambiamos 'nombre_servicio' a 'nom_servicio' y le agregamos 'nombre_paquete' a PAQUETES.

Entonces, nos queda:

Clientes(num_cliente CHAR(8), Nombres_c VARCHAR(30),
Apellido_Paterno_c VARCHAR(30), Apellido_Materno_c VARCHAR(30),
metodo_pago VARCHAR(14), paquete_c VARCHAR(30), fecha_c DATE)

Paquetes(id_paquete CHAR(8))

Servicios(id_servicio CHAR(8), nom_servicio VARCHAR(30))

Empleados(Num_empleado CHAR(8), Nombres_E VARCHAR(30),
Apellido_Paterno_E VARCHAR(30), Apellido_Materno_E VARCHAR(30),
Categoria_E VARCHAR(30))

Camas_Solares(Num_Serie CHAR(8), Num_Focos INTEGER, Fecha_uso DATE)

Instalaciones(Id_Instalacion CHAR(8), Num_Camas INTEGER)

Contratan(num_clientes CHAR(8), id_paquete CHAR(8))

Incluyen(id_paquete CHAR(8), id_servicio CHAR(8))

Realizan(id_servicio CHAR(8), Num_empleado CHAR(8), Fecha DATE)

Ofrecen(id_servicio CHAR(8), Id_Secursal CHAR(8), Num_Serie VARCHAR(8))

2FN: Identificamos una dependencia parcial: 'Fecha_uso' en Camas_Solares no depende de qué cama específica se usó, por lo que se eliminará ese atributo y sólo se conservará 'fecha_r' en Realizan para llevar un control de cuándo se realiza un servicio, por quién y dónde. Además, 'fecha_r' depende de cuándo un empleado le realiza cierto servicio a un cliente por lo que dentro de esa tabla podríamos tener múltiples registros del mismo empleado y cliente, pero en diferentes fechas por eso se hace parte de la llave primaria compuesta.

Clientes(num_cliente CHAR(8), Nombres_c VARCHAR(30),
Apellido_Paterno_c VARCHAR(30), Apellido_Materno_c VARCHAR(30),
metodo_pago VARCHAR(14), paquete_c VARCHAR(30), fecha_c DATE)

Paquetes(id_paquete CHAR(8))

Servicios(id_servicio CHAR(8), nom_servicio VARCHAR(30))

Empleados(Num_empleado CHAR(8), Nombres_E VARCHAR(30),
Apellido_Paterno_E VARCHAR(30), Apellido_Materno_E VARCHAR(30),
Categoria_E VARCHAR(30))

Camas_Solares(Num_Serie CHAR(8), Num_Focos INTEGER)

Instalaciones(Id_Instalacion CHAR(8), Num_Camas INTEGER)

Contratan(num_clientes CHAR(8), id_paquete CHAR(8))

Incluyen(id_paquete CHAR(8), id_servicio CHAR(8))

Realizan(id_servicio CHAR(8), Num_empleado CHAR(8), Fecha_r DATE)

Ofrecen(id_servicio CHAR(8), Id_Secursal CHAR(8), Num_Serie VARCHAR(8))

3FN: Finalmente, identificamos las dependencias transitivas: notamos que 'paquete_c', que representa el nombre del paquete contratado por el cliente no depende del cliente sino de Paquetes y le cambiamos el nombre a 'nombre_paquete'.

Observamos que en Cliente con 'fecha_c' que representa la fecha de contratación de un paquete no depende de qué cliente lo contrató, sino de la relación CONTRATAN. Además, 'fecha_c' depende directamente de cuándo un cliente contrata cierto paquete y podría darse el caso donde se contrata el mismo paquete en diferentes fechas, por lo que 'fecha_c' debe agregarse a la llave primaria compuesta de CONTRATAN.

También notamos que 'metodo_pago' no depende de cliente sino de la relación CONTRATAN.

Clientes(num_cliente CHAR(8), Nombres_c VARCHAR(30),
Apellido_Paterno_c VARCHAR(30), Apellido_Materno_c VARCHAR(30))

Paquetes(id_paquete CHAR(8), Nombre_paquete VARCHAR(30))

Servicios(id_servicio CHAR(8), nom_servicio VARCHAR(30))

Empleados(Num_empleado CHAR(8), Nombres_E VARCHAR(30),
Apellido_Paterno_E VARCHAR(30), Apellido_Materno_E VARCHAR(30),
Categoria_E VARCHAR(30))

Camas_Solares(Num_Serie CHAR(8), Num_Focos INTEGER)

Instalaciones(Id_Instalacion CHAR(8), Num_Camas INTEGER)

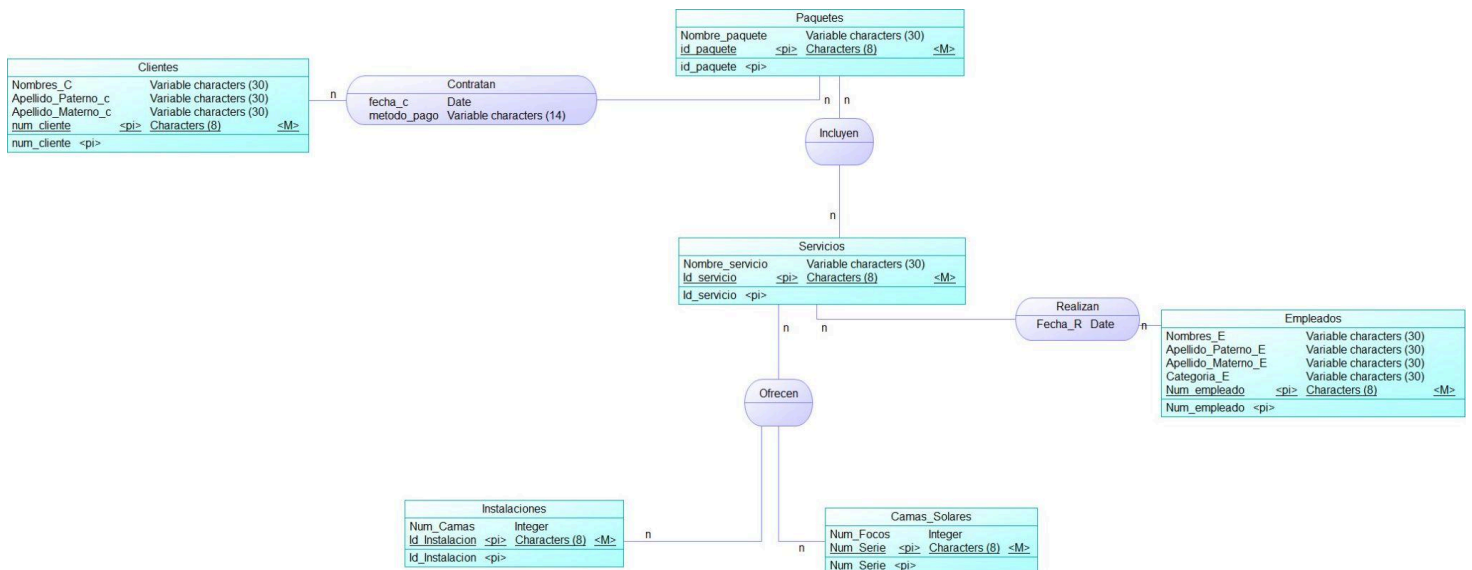
Contratan(num_clientes CHAR(8), id_paquete CHAR(8), fecha_c DATE, metodo_pago VARCHAR(14))

Incluyen(id_paquete CHAR(8), id_servicio CHAR(8))

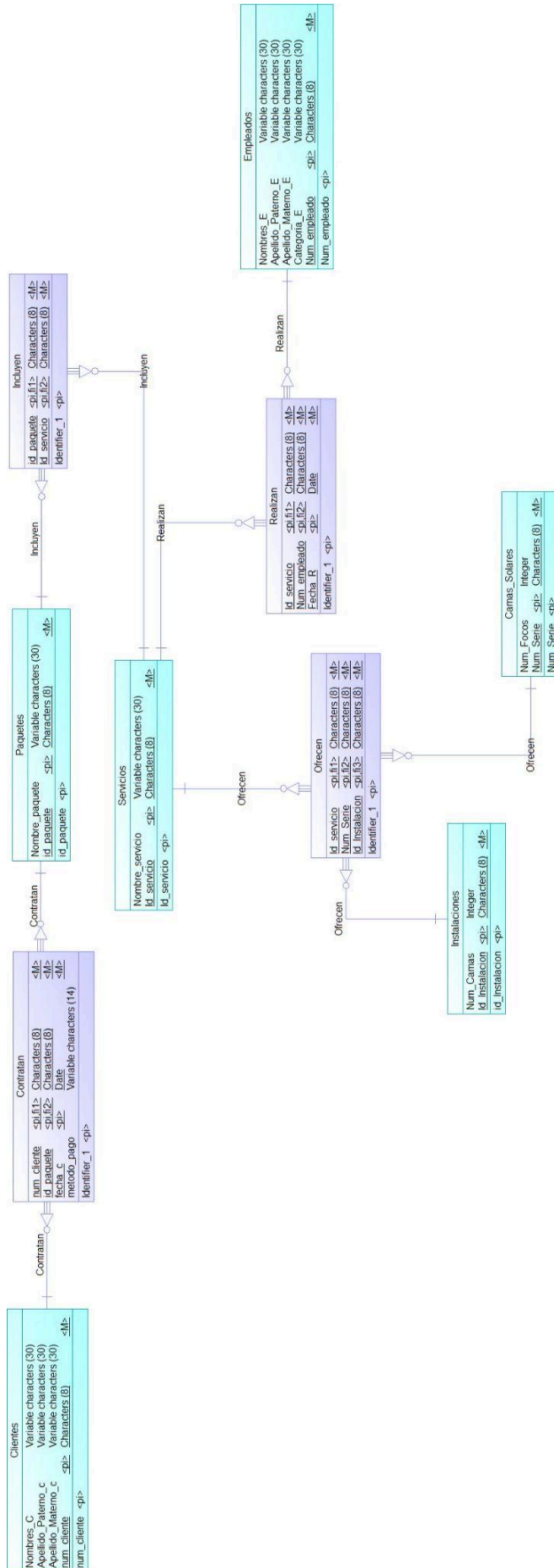
Realizan(id_servicio CHAR(8), Num_empleado CHAR(8), Fecha_r DATE)

Ofrecen(id_servicio CHAR(8), Id_Secursal CHAR(8), Num_Serie VARCHAR(8))

Modelo Entidad-Relación Normalizado



Modelo Relacional Normalizado



Creación de la base de datos con el nombre que se indica en el encabezado.

```
[postgres=> CREATE DATABASE SPA;
CREATE DATABASE
[postgres=> \c spa;
Ahora está conectado a la base de datos «spa» con el usuario «alumno06».
spa=> █
```

reclusos_14	alumno28	UTF8	es_MX.UTF-8	es_MX.UTF-8	
spa	alumno06	UTF8	es_MX.UTF-8	es_MX.UTF-8	
supermercadowcodenieve	postgres	UTF8	es_MX.UTF-8	es_MX.UTF-8	=Tc/postgres +
					postgres=CTc/postgres+
					alumnos=Tc/postgres
template0	postgres	UTF8	es_MX.UTF-8	es_MX.UTF-8	=c/postgres +

Creación de tablas.

```
cursodbe=> \c spa;
Ahora está conectado a la base de datos «spa» con el usuario «alumno06».
spa=> CREATE TABLE Clientes (
        num_cliente CHAR(8),
        nombres_c VARCHAR(30),
        apellido_paterno_c VARCHAR(30),
        apellido_materno_c VARCHAR(30)
);
CREATE TABLE
```

```
milenafer — alumno06@master:~ — ssh -p2206 alumno06@132.248.51.117 — 134x58
-- Conectarse a la base de datos
\c spa;

-- Crear tabla Paquetes
CREATE TABLE Paquetes (
        id_paquete CHAR(8),
        nombre_paquete VARCHAR(30)
);

-- Crear tabla Servicios
CREATE TABLE Servicios (
        id_servicio CHAR(8),
        nombre_servicio VARCHAR(30)
);

-- Crear tabla Empleados
CREATE TABLE Empleados (
        num_empleado CHAR(8),
        nombres_e VARCHAR(30),
        apellido_paterno_c VARCHAR(30),
        apellido_materno_c VARCHAR(30),
        categoria_e VARCHAR(30)
);

-- Crear tabla Camas_Solares
CREATE TABLE Camas_Solares (
        num_serie CHAR(8),
        num_focos INTEGER
);

-- Crear tabla Instalaciones
CREATE TABLE Instalaciones (
        id_instalacion CHAR(8),
        num_camas INTEGER
);

-- Crear tabla de relacion Contratan
CREATE TABLE Contratan (
        num_cliente CHAR(8),
        id_paquete CHAR(8),
        fecha_c DATE,
        metodo_pago VARCHAR(13)
);

-- Crear tabla de relación Ofrecen
CREATE TABLE Ofrecen (
        id_servicio CHAR(8),
        num_serie CHAR(8),
        id_instalacion CHAR(8)
);

-- Crear tabla de relacion Incluyen
CREATE TABLE Incluyen (
        id_paquete CHAR(8),
        id_servicio CHAR(8)
);

-- Crear tabla de relacion Realizan
CREATE TABLE Realizan (
        id_servicio CHAR(8),
        num_empleado CHAR(8),
        fecha_r DATE
);

1,1 Comienzo

122,0-1 Final
```



```
[alumno06@master ~]$ vi tablas_spa
[alumno06@master ~]$ psql -d postgres
psql (13.20)
Digite «help» para obtener ayuda.
```

```
[postgres=> \i tablas_spa
Ahora está conectado a la base de datos «spa» con el usuario «alumno06».
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
```

```
[spa=> -d spa -c \dt
Listado de relaciones
Esquema | Nombre | Tipo | Dueño
-----+-----+-----+-----
public | camas_solares | tabla | alumno06
public | clientes | tabla | alumno06
public | contratan | tabla | alumno06
public | empleados | tabla | alumno06
public | incluyen | tabla | alumno06
public | instalaciones | tabla | alumno06
public | ofrecen | tabla | alumno06
public | paquetes | tabla | alumno06
public | realizan | tabla | alumno06
public | servicios | tabla | alumno06
(10 filas)

spa-> █
```

Creación de llaves foráneas y primarias según corresponda con el modelo lógico.

```
Conectarse a la base de datos
\c spa;

-- Crear llave primaria Clientes
ALTER TABLE Clientes
ADD CONSTRAINT pk_clientes
PRIMARY KEY (num_cliente);

-- Crear llave primaria Paquetes
ALTER TABLE Paquetes
ADD CONSTRAINT pk_paquetes
PRIMARY KEY (id_paquete);

-- Crear llave primaria Servicio
ALTER TABLE Servicio
ADD CONSTRAINT pk_servicio
PRIMARY KEY (id_servicio);

-- Crear llave primaria Empleado
ALTER TABLE Empleado
ADD CONSTRAINT pk_empleado
PRIMARY KEY (num_empleado);

-- Crear llave primaria Camas_Solares
ALTER TABLE Camas_Solares
ADD CONSTRAINT pk_camas_solares
PRIMARY KEY (num_serie);

-- Crear llave primaria Instalaciones
ALTER TABLE Instalaciones
ADD CONSTRAINT pk_instalaciones
PRIMARY KEY (id_instalacion);

-- Crear llave foranea Contratan
ALTER TABLE Contratan
ADD CONSTRAINT pk_contratan
PRIMARY KEY (num_cliente, id_paquete);

ALTER TABLE Contratan
ADD CONSTRAINT fk_contratan_cliente
FOREIGN KEY (num_cliente) REFERENCES Clientes(num_cliente)
;

ALTER TABLE Contratan
ADD CONSTRAINT fk_contratan_paquete
FOREIGN KEY (id_paquete) REFERENCES Paquetes(id_paquete)
;

-- Crear llave foranea Incluyen
ALTER TABLE Incluyen
ADD CONSTRAINT pk_incluyen
PRIMARY KEY (id_paquete,id_servicio);

ALTER TABLE Incluyen
ADD CONSTRAINT fk_incluyen_paquetes
FOREIGN KEY (id_paquete) REFERENCES Paquetes(id_paquete)
;

-- INSERTAR --
```

1,1 Comienzo

```

milenafer — alumno06@master:~ — ssh -p2206 alumno06@132.248.51.117 — 134x58
FOREIGN KEY (id_paquete) REFERENCES Paquetes(id_paquete)
;
-- Crear llave foranea Incluyen
ALTER TABLE Incluyen
ADD CONSTRAINT pk_incluyen
PRIMARY KEY (id_paquete,id_servicio);

ALTER TABLE Incluyen
ADD CONSTRAINT fk_incluyen_paquetes
FOREIGN KEY (id_paquete) REFERENCES Paquetes(id_paquete)
;

ALTER TABLE Incluyen
ADD CONSTRAINT fk_incluyen_servicios
FOREIGN KEY (id_servicio) REFERENCES Servicios(id_servicio)
;

---Crear llave foranea Realizan

-- Llave primaria compuesta de Realizan
ALTER TABLE Realizan
ADD CONSTRAINT pk_realizan
PRIMARY KEY (id_servicio, num_empleado);

ALTER TABLE Realizan
ADD CONSTRAINT fk_realizan_servicio
FOREIGN KEY (id_servicio) REFERENCES Servicios(id_servicio)
;

ALTER TABLE Realizan
ADD CONSTRAINT fk_realizan_empleado
FOREIGN KEY (num_empleado) REFERENCES Empleados(num_empleado)
;

---Crear llave foranea Ofrecen

ALTER TABLE Ofrecen
ADD CONSTRAINT pk_ofrecen
PRIMARY KEY (id_servicio, num_serie, id_instalacion);

ALTER TABLE Ofrecen
ADD CONSTRAINT fk_ofrecen_servicio
FOREIGN KEY (id_servicio) REFERENCES Servicios(id_servicio)
;

ALTER TABLE Ofrecen
ADD CONSTRAINT fk_ofrecen_cama
FOREIGN KEY (num_serie) REFERENCES Camas_Solares(num_serie)
;

ALTER TABLE Ofrecen
ADD CONSTRAINT fk_ofrecen_instalacion
FOREIGN KEY (id_instalacion) REFERENCES Instalaciones(id_instalacion)
;

-- INSERTAR --
103,1      Final

```

```

[alumno06@master ~]$ vi llavespk
[alumno06@master ~]$ psql -d postgres
psql (13.20)
Digite «help» para obtener ayuda.

postgres=> \i llavespk
Ahora está conectado a la base de datos «spa» con el usuario «alumno06».
ALTER TABLE
ALTER TABLE
psql:llavespk:17: ERROR: no existe la relación «servicio»
psql:llavespk:22: ERROR: no existe la relación «empleado»
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
ALTER TABLE
psql:llavespk:62: ERROR: no hay restricción unique que coincida con las columnas dadas en la tabla referida «servicios»
ALTER TABLE
psql:llavespk:74: ERROR: no hay restricción unique que coincida con las columnas dadas en la tabla referida «servicios»
psql:llavespk:79: ERROR: no hay restricción unique que coincida con las columnas dadas en la tabla referida «empleados»
ALTER TABLE
psql:llavespk:90: ERROR: no hay restricción unique que coincida con las columnas dadas en la tabla referida «servicios»
ALTER TABLE
ALTER TABLE
spa=>

```

NOTA: Los errores los corregimos en las siguientes líneas.

```
spa=> ALTER TABLE Servicios
ADD CONSTRAINT pk_servicio
PRIMARY KEY (id_servicio);
ALTER TABLE
spa=> ALTER TABLE Empleados
ADD CONSTRAINT pk_empleado
PRIMARY KEY (num_empleado);
ALTER TABLE
```

```
spa=> ALTER TABLE Incluyen
ADD CONSTRAINT fk_incluyen_servicios
FOREIGN KEY (id_servicio) REFERENCES Servicios(id_servicio)
;
ALTER TABLE
spa=> ALTER TABLE Realizan
ADD CONSTRAINT fk_realizan_servicio
FOREIGN KEY (id_servicio) REFERENCES Servicios(id_servicio)
;
ALTER TABLE
spa=> ALTER TABLE Realizan
ADD CONSTRAINT fk_realizan_empleado
FOREIGN KEY (num_empleado) REFERENCES Empleados(num_empleado)
;
ALTER TABLE
```

```
spa=> ALTER TABLE Ofrecen
ADD CONSTRAINT fk_ofrecen_servicio
FOREIGN KEY (id_servicio) REFERENCES Servicios(id_servicio)
;
ALTER TABLE
```

Modificamos los siguientes constraints porque nos dimos cuenta de que nos faltó 'fecha_r' en la llave primaria compuesta de REALIZANy 'fecha_c' en la llave primaria de CONTRATAN, como habíamos definido en la 3FN.

```
spa=> ALTER TABLE realizan DROP CONSTRAINT IF EXISTS pk_realizan;
ALTER TABLE
spa=> ALTER TABLE Realizan
ADD CONSTRAINT pk_realizan
PRIMARY KEY (id_servicio, num_empleado, fecha_r);
ALTER TABLE
```



```
[spa=> ALTER TABLE contratan DROP CONSTRAINT IF EXISTS pk_contratan;
ALTER TABLE
[spa=> ALTER TABLE contratan ADD CONSTRAINT pk_contratan PRIMARY KEY (num_cliente,
id_paquete, fecha_c);
ALTER TABLE
```

Así quedaron las tablas:

```
spa=> \d clientes

          Tabla «public.clientes»
  Columna | Tipo | Ordenamiento | Nulable | Por omisión
-----+-----+-----+-----+-----
 num_cliente | character(8) | | not null |
 nombres_c | character varying(30) | | |
 apellido_paterno_c | character varying(30) | | |
 apellido_materno_c | character varying(30) | | |
Índices:
 "pk_clientes" PRIMARY KEY, btree (num_cliente)
Referenciada por:
 TABLE "contratan" CONSTRAINT "fk_contratan_cliente" FOREIGN KEY (num_cliente) REFERENCES clientes(num_cliente)

spa=>
```

```
spa=> \d paquetes

          Tabla «public.paquetes»
  Columna | Tipo | Ordenamiento | Nulable | Por omisión
-----+-----+-----+-----+-----
 id_paquete | character(8) | | not null |
 nombre_paquete | character varying(30) | | |
Índices:
 "pk_paquetes" PRIMARY KEY, btree (id_paquete)
Referenciada por:
 TABLE "contratan" CONSTRAINT "fk_contratan_paquete" FOREIGN KEY (id_paquete) REFERENCES paquetes(id_paquete)
 TABLE "incluyen" CONSTRAINT "fk_incluyen_paquetes" FOREIGN KEY (id_paquete) REFERENCES paquetes(id_paquete)

spa=>
```

```
spa=> \d servicios

          Tabla «public.servicios»
  Columna | Tipo | Ordenamiento | Nulable | Por omisión
-----+-----+-----+-----+-----
 id_servicio | character(8) | | not null |
 nombre_servicio | character varying(30) | | |
Índices:
 "pk_servicio" PRIMARY KEY, btree (id_servicio)
Referenciada por:
 TABLE "incluyen" CONSTRAINT "fk_incluyen_servicios" FOREIGN KEY (id_servicio) REFERENCES servicios(id_servicio)
 TABLE "ofrecen" CONSTRAINT "fk_ofrecen_servicio" FOREIGN KEY (id_servicio) REFERENCES servicios(id_servicio)
 TABLE "realizan" CONSTRAINT "fk_realizan_servicio" FOREIGN KEY (id_servicio) REFERENCES servicios(id_servicio)

spa=>
```

```
spa=> \d empleados

          Tabla «public.empleados»
  Columna | Tipo | Ordenamiento | Nulable | Por omisión
-----+-----+-----+-----+-----
 num_empleado | character(8) | | not null |
 nombres_e | character varying(30) | | |
 apellido_paterno_e | character varying(30) | | |
 apellido_materno_e | character varying(30) | | |
 categoria_e | character varying(30) | | |
Índices:
 "pk_empleado" PRIMARY KEY, btree (num_empleado)
Referenciada por:
 TABLE "realizan" CONSTRAINT "fk_realizan_empleado" FOREIGN KEY (num_empleado) REFERENCES empleados(num_empleado)

spa=>
```

```
spa=> \d camas_solares

          Tabla «public.camas_solares»
  Columna | Tipo | Ordenamiento | Nulable | Por omisión
-----+-----+-----+-----+-----
 num_serie | character(8) | | not null |
 num_focos | integer | | |
Índices:
 "pk_camas_solares" PRIMARY KEY, btree (num_serie)
Referenciada por:
 TABLE "ofrecen" CONSTRAINT "fk_ofrecen_cama" FOREIGN KEY (num_serie) REFERENCES camas_solares(num_serie)

spa=>
```

```
spa=> \d instalaciones
      Tabla «public.instalaciones»
  Columna | Tipo | Ordenamiento | Nulable | Por omisión
-----+-----+-----+-----+-----
id_instalacion | character(8) | | not null |
num_camras | integer | | |
Índices:
    "pk_instalaciones" PRIMARY KEY, btree (id_instalacion)
Referenciada por:
    TABLE "ofrecen" CONSTRAINT "fk_ofrecen_instalacion" FOREIGN KEY (id_instalacion) REFERENCES instalaciones(id_instalacion)
```

```
spa=> \d contratan
      Tabla «public.contratan»
  Columna | Tipo | Ordenamiento | Nulable | Por omisión
-----+-----+-----+-----+-----
num_cliente | character(8) | | not null |
id_paquete | character(8) | | not null |
fecha_c | date | | |
metodo_pago | character varying(13) | | |
Índices:
    "pk_contratan" PRIMARY KEY, btree (num_cliente, id_paquete, fecha_c)
Restricciones de llave foránea:
    "fk_contratan_cliente" FOREIGN KEY (num_cliente) REFERENCES clientes(num_cliente)
    "fk_contratan_paquete" FOREIGN KEY (id_paquete) REFERENCES paquetes(id_paquete)
```

NOTA: Posteriormente, modificamos el tipo de dato de 'metodo_pago' de VARCHAR(13) a VARCHAR(14) con la siguiente instrucción:

```
spa=> ALTER TABLE Contratan
[ALTER COLUMN metodo_pago TYPE VARCHAR(14);
ALTER TABLE
```

```
spa=> \d contratan
      Tabla «public.contratan»
  Columna | Tipo | Ordenamiento | Nulable | Por omisión
-----+-----+-----+-----+-----
num_cliente | character(8) | | not null |
id_paquete | character(8) | | not null |
fecha_c | date | | |
metodo_pago | character varying(14) | | not null |
Índices:
    "pk_contratan" PRIMARY KEY, btree (num_cliente, id_paquete)
Restricciones de llave foránea:
    "fk_contratan_cliente" FOREIGN KEY (num_cliente) REFERENCES clientes(num_cliente)
    "fk_contratan_paquete" FOREIGN KEY (id_paquete) REFERENCES paquetes(id_paquete)
```

```
spa=> \d ofrecen
      Tabla «public.ofrecen»
  Columna | Tipo | Ordenamiento | Nulable | Por omisión
-----+-----+-----+-----+-----
id_servicio | character(8) | | not null |
num_serie | character(8) | | not null |
id_instalacion | character(8) | | not null |
Índices:
    "pk_ofrecen" PRIMARY KEY, btree (id_servicio, num_serie, id_instalacion)
Restricciones de llave foránea:
    "fk_ofrecen_cama" FOREIGN KEY (num_serie) REFERENCES camas_solares(num_serie)
    "fk_ofrecen_instalacion" FOREIGN KEY (id_instalacion) REFERENCES instalaciones(id_instalacion)
    "fk_ofrecen_servicio" FOREIGN KEY (id_servicio) REFERENCES servicios(id_servicio)
```

```
[spa=> \d incluyen
```

```
          Tabla «public.incluyen»
  Columna | Tipo | Ordenamiento | Nulable | Por omisión
-----+-----+-----+-----+-----
id_paquete | character(8) | | not null |
id_servicio | character(8) | | not null |
Índices:
    "pk_incluyen" PRIMARY KEY, btree (id_paquete, id_servicio)
Restricciones de llave foránea:
    "fk_incluyen_paquetes" FOREIGN KEY (id_paquete) REFERENCES paquetes(id_paquete)
    "fk_incluyen_servicios" FOREIGN KEY (id_servicio) REFERENCES servicios(id_servicio)
```

```
[spa=> \d realizan
```

```
          Tabla «public.realizan»
  Columna | Tipo | Ordenamiento | Nulable | Por omisión
-----+-----+-----+-----+-----
id_servicio | character(8) | | not null |
num_empleado | character(8) | | not null |
fecha_r | date | | not null |
Índices:
    "pk_realizan" PRIMARY KEY, btree (id_servicio, num_empleado, fecha_r)
Restricciones de llave foránea:
    "fk_realizan_empleado" FOREIGN KEY (num_empleado) REFERENCES empleados(num_empleado)
    "fk_realizan_servicio" FOREIGN KEY (id_servicio) REFERENCES servicios(id_servicio)
```

```
[spa=>
```


Creación de reglas, constraints o defaults que permitan integridad de dominio y/o reglas de negocio.

Para agregarle constraints a foreign keys es necesario hacerlo desde la definición de éstas, por lo que debemos eliminar las que habíamos definido en el paso anterior y darlas de alta nuevamente con las restricciones.

```
spa=> ALTER TABLE clientes
ALTER COLUMN nombres_c SET NOT NULL, -- siempre existe nombre
ALTER COLUMN apellido_paterno_c SET NOT NULL, -- siempre tienes al menos un apellido
ADD CONSTRAINT nombres_sololetras
CHECK (nombres_c ~ '^[A-ZÁÉÍÓÚÑa-záéíóúñ\s]+$'), -- solo letras
ADD CONSTRAINT apellido_sololetras
CHECK (apellido_paterno_c ~ '^[A-ZÁÉÍÓÚÑa-záéíóúñ]+$');
ALTER TABLE
```

```
spa=> ALTER TABLE paquetes
[ALTER COLUMN nombre_paquete SET NOT NULL;
ALTER TABLE
```

```
spa=> ALTER TABLE servicios
[ALTER COLUMN nombre_servicio SET NOT NULL;
ALTER TABLE
```

```
spa=> ALTER TABLE empleados
ALTER COLUMN nombres_e SET NOT NULL, -- siempre tienes nombre
ALTER COLUMN apellido_paterno_e SET NOT NULL, -- siempre tienes al menos un apellido
ALTER COLUMN categoria_e SET NOT NULL, -- siempre trabajas en algo específico
ADD CONSTRAINT categoria_employado -- debe existir la categoría de lo que trabajas
CHECK (categoria_e IN ('Recepcionista', 'Terapeuta', 'Masajista', 'Esteticista', 'Gerente')),
ADD CONSTRAINT nombre_sololetras
CHECK (nombres_e ~ '^[A-ZÁÉÍÓÚÑa-záéíóúñ\s]+$'), -- solo letras
ADD CONSTRAINT apellido_sololetras
CHECK (apellido_paterno_e ~ '^[A-ZÁÉÍÓÚÑa-záéíóúñ]+$');
ALTER TABLE
```

```
spa=> ALTER TABLE camas_solares
ADD CONSTRAINT camas_num_focos -- siempre hay num focos positivo (al menos 1)
[CHECK (num_focos > 0);
ALTER TABLE
```

```
spa=> ALTER TABLE instalaciones
ADD CONSTRAINT instalaciones_num_camas
[CHECK (num_camas > 0); -- siempre debe haber al menos 1 cama
ALTER TABLE
```

```
spa=> ALTER TABLE contratan
[ALTER COLUMN fecha_c SET DEFAULT CURRENT_DATE; -- si no meten fecha se asume que ocurrió hoy
ALTER TABLE
```



```
spa=> ALTER TABLE contratan
ALTER COLUMN fecha_c SET NOT NULL, -- contratos deben tener fecha siempre
ALTER COLUMN metodo_pago SET NOT NULL, -- y debe pagarse de alguna forma establecida
DROP CONSTRAINT IF EXISTS contratan_metodo_pago,
ADD CONSTRAINT contratan_metodo_pago
CHECK (metodo_pago IN ('Efectivo', 'TarjetaCrédito', 'TarjetaDébito', 'Transferencia', 'Depósito'));
ALTER TABLE
spa=>
```

```
[ADD CONSTRAINT contratan_unico UNIQUE (num_cliente, id_paquete, fecha_c); -- no debe haber contratos duplicados con misma fecha
ALTER TABLE
```

```
spa=> ALTER TABLE contratan
DROP CONSTRAINT IF EXISTS fk_contratan_cliente,
ADD CONSTRAINT fk_contratan_cliente
FOREIGN KEY (num_cliente) REFERENCES Clientes(num_cliente)
[ON DELETE CASCADE;
ALTER TABLE
```

```
spa=> ALTER TABLE contratan
DROP CONSTRAINT IF EXISTS fk_contratan_paquete,
ADD CONSTRAINT fk_contratan_paquete
FOREIGN KEY (id_paquete) REFERENCES Paquetes(id_paquete)
ON DELETE RESTRICT;
ALTER TABLE
```

```
spa=> ALTER TABLE ofrecen
DROP CONSTRAINT IF EXISTS fk_ofrecen_servicio,
ADD CONSTRAINT fk_ofrecen_servicio
FOREIGN KEY (id_servicio)
REFERENCES Servicios(id_servicio)
ON DELETE CASCADE; -- si eliminamos Servicio deja de estar disponible para ofertarse entonces eliminamos todos los registros relacionados
ALTER TABLE
```

```
spa=> ALTER TABLE ofrecen
DROP CONSTRAINT IF EXISTS fk_ofrecen_cama,
ADD CONSTRAINT fk_ofrecen_cama
FOREIGN KEY (num_serie)
REFERENCES Camas_Solares(num_serie)
ON DELETE CASCADE; -- si deja de servir una cama no nos interesa historial de cuantas veces se ha ocupado, simplemente se deja de ser un activo de la empresa
ALTER TABLE
```

```
spa=> ALTER TABLE ofrecen
DROP CONSTRAINT IF EXISTS fk_ofrecen_instalacion,
ADD CONSTRAINT fk_ofrecen_instalacion
FOREIGN KEY (id_instalacion)
REFERENCES Instalaciones(id_instalacion)
ON DELETE CASCADE; -- si un spa cierra ya no forma parte de la empresa entonces no nos interesa conservar el historial de todo lo que ofrecio
ALTER TABLE
```

```
spa=> ALTER TABLE incluyen
ADD CONSTRAINT fk_incluyen_servicio
FOREIGN KEY (id_servicio) REFERENCES Servicios(id_servicio)
ON DELETE RESTRICT; -- no podemos eliminar servicios que aun se ofertan en paquetes
ALTER TABLE
```

```
spa=> ALTER TABLE realizan
DROP CONSTRAINT IF EXISTS fk_realizan_empleado,
ADD CONSTRAINT fk_realizan_empleado
FOREIGN KEY (num_empleado)
REFERENCES Empleados(num_empleado)
ON DELETE CASCADE; -- si empleado renuncia no queremos conservar sus registros de los servicios que realizo, deja de formar parte del equipo de trabajo y de estar disponible para realizar servicios.
ALTER TABLE
```

```
spa=> ALTER TABLE realizan
ALTER COLUMN fecha_r SET DEFAULT CURRENT_DATE; -- sino meten fecha se asume que es hoy
ALTER TABLE
```

Así quedaron las tablas:

```
spa=> \d clientes

          Tabla «public.clientes»
+-----+-----+-----+-----+-----+
| Columna | Tipo | Ordenamiento | Nulable | Por omisión |
+-----+-----+-----+-----+-----+
| num_cliente | character(8) | | not null | |
| nombres_c | character varying(30) | | not null | |
| apellido_paterno_c | character varying(30) | | not null | |
| apellido_materno_c | character varying(30) | | | |
+-----+-----+-----+-----+-----+
Índices:
"pk_clientes" PRIMARY KEY, btree (num_cliente)
Restricciones CHECK:
"apellido_paterno_c" CHECK (apellido_paterno_c::text ~ '^[A-ZÁÉÍÓÚÑá-záéíóúñ]+$'::text)
"nombres_c" CHECK (nombres_c::text ~ '^[A-ZÁÉÍÓÚÑá-záéíóúñ\s]+$'::text)
Referenciada por:
TABLE "contratan" CONSTRAINT "fk_contratan_cliente" FOREIGN KEY (num_cliente) REFERENCES clientes(num_cliente) ON DELETE CASCADE
```

```
spa=> \d paquetes

          Tabla «public.paquetes»
+-----+-----+-----+-----+-----+
| Columna | Tipo | Ordenamiento | Nulable | Por omisión |
+-----+-----+-----+-----+-----+
| id_paquete | character(8) | | not null | |
| nombre_paquete | character varying(30) | | not null | |
+-----+-----+-----+-----+-----+
Índices:
"pk_paquetes" PRIMARY KEY, btree (id_paquete)
Referenciada por:
TABLE "contratan" CONSTRAINT "fk_contratan_paquete" FOREIGN KEY (id_paquete) REFERENCES paquetes(id_paquete) ON DELETE RESTRICT
TABLE "incluyen" CONSTRAINT "fk_incluyen_paquetes" FOREIGN KEY (id_paquete) REFERENCES paquetes(id_paquete)
```

```
spa=> \d servicios

          Tabla «public.servicios»
+-----+-----+-----+-----+-----+
| Columna | Tipo | Ordenamiento | Nulable | Por omisión |
+-----+-----+-----+-----+-----+
| id_servicio | character(8) | | not null | |
| nombre_servicio | character varying(30) | | not null | |
+-----+-----+-----+-----+-----+
Índices:
"pk_servicio" PRIMARY KEY, btree (id_servicio)
Referenciada por:
TABLE "incluyen" CONSTRAINT "fk_incluyen_servicio" FOREIGN KEY (id_servicio) REFERENCES servicios(id_servicio) ON DELETE RESTRICT
TABLE "incluyen" CONSTRAINT "fk_incluyen_servicios" FOREIGN KEY (id_servicio) REFERENCES servicios(id_servicio)
TABLE "ofrecen" CONSTRAINT "fk_ofrecen_servicio" FOREIGN KEY (id_servicio) REFERENCES servicios(id_servicio) ON DELETE CASCADE
TABLE "realizan" CONSTRAINT "fk_realizan_servicio" FOREIGN KEY (id_servicio) REFERENCES servicios(id_servicio)
```

```
spa=> \d empleados

          Tabla «public.empleados»
+-----+-----+-----+-----+-----+
| Columna | Tipo | Ordenamiento | Nulable | Por omisión |
+-----+-----+-----+-----+-----+
| num_empleado | character(8) | | not null | |
| nombres_e | character varying(30) | | not null | |
| apellido_paterno_e | character varying(30) | | not null | |
| apellido_materno_e | character varying(30) | | | |
| categoria_e | character varying(30) | | not null | |
+-----+-----+-----+-----+-----+
Índices:
"pk_empleado" PRIMARY KEY, btree (num_empleado)
"idx_empleados_nombre" btree (nombres_e, apellido_paterno_e, apellido_materno_e)
Restricciones CHECK:
"apellido_paterno_e" CHECK (apellido_paterno_e::text ~ '^[A-ZÁÉÍÓÚÑá-záéíóúñ]+$'::text)
"categoria_empleado" CHECK (categoria_e::text = ANY (ARRAY['Recepcionista'::character varying, 'Terapeuta'::character varying, 'Masajista'::character varying, 'Esteticista'::character varying, 'Gerente'::character varying]))
"nombre_empleado" CHECK (nombres_e::text ~ '^[A-ZÁÉÍÓÚÑá-záéíóúñ\s]+$'::text)
Referenciada por:
TABLE "realizan" CONSTRAINT "fk_realizan_empleado" FOREIGN KEY (num_empleado) REFERENCES empleados(num_empleado) ON DELETE CASCADE
```



```

spa=> \d camas_solares
          Tabla «public.camas_solares»
  Columna |      Tipo      | Ordenamiento | Nulable | Por omisión
-----+-----+-----+-----+-----
 num_serie | character(8) |              | not null | 
 num_focos | integer      |              |          | 
Índices:
 "pk_camas_solares" PRIMARY KEY, btree (num_serie)
Restricciones CHECK:
 "camas_num_focos" CHECK (num_focos > 0)
Referenciada por:
 TABLE "ofrecen" CONSTRAINT "fk_ofrecen_cama" FOREIGN KEY (num_serie) REFERENCES camas_solares(num_serie) ON DELETE CASCADE

```

```

spa=> \d instalaciones
          Tabla «public.instalaciones»
  Columna |      Tipo      | Ordenamiento | Nulable | Por omisión
-----+-----+-----+-----+-----
 id_instalacion | character(8) |              | not null | 
 num_camas      | integer      |              |          | 
Índices:
 "pk_instalaciones" PRIMARY KEY, btree (id_instalacion)
Restricciones CHECK:
 "instalaciones_num_camas" CHECK (num_camas > 0)
Referenciada por:
 TABLE "ofrecen" CONSTRAINT "fk_ofrecen_instalacion" FOREIGN KEY (id_instalacion) REFERENCES instalaciones(id_instalacion) ON DELETE CASCADE

```

```

spa=> \d contratan
          Tabla «public.contratan»
  Columna |      Tipo      | Ordenamiento | Nulable | Por omisión
-----+-----+-----+-----+-----
 num_cliente | character(8) |              | not null | 
 id_paquete  | character(8) |              | not null | 
 fecha_c     | date         |              | not null | CURRENT_DATE
 metodo_pago | character varying(14) |              | not null | 
Índices:
 "pk_contratan" PRIMARY KEY, btree (num_cliente, id_paquete, fecha_c)
 "contratan_unico" UNIQUE CONSTRAINT, btree (num_cliente, id_paquete, fecha_c)
Restricciones CHECK:
 "contratan_metodo_pago" CHECK (metodo_pago::text = ANY (ARRAY['Efectivo'::character varying::text, 'TarjetaCrédito'::character varying::text, 'TarjetaDébito'::character varying::text, 'Transferencia'::character varying::text, 'Depósito'::character varying::text]))
Restricciones de llave foránea:
 "fk_contratan_cliente" FOREIGN KEY (num_cliente) REFERENCES clientes(num_cliente) ON DELETE CASCADE
 "fk_contratan_paquete" FOREIGN KEY (id_paquete) REFERENCES paquetes(id_paquete) ON DELETE RESTRICT

```

```

spa=> \d ofrecen
          Tabla «public.ofrecen»
  Columna |      Tipo      | Ordenamiento | Nulable | Por omisión
-----+-----+-----+-----+-----
 id_servicio | character(8) |              | not null | 
 num_serie   | character(8) |              | not null | 
 id_instalacion | character(8) |              | not null | 
Índices:
 "pk_ofrecen" PRIMARY KEY, btree (id_servicio, num_serie, id_instalacion)
Restricciones de llave foránea:
 "fk_ofrecen_cama" FOREIGN KEY (num_serie) REFERENCES camas_solares(num_serie) ON DELETE CASCADE
 "fk_ofrecen_instalacion" FOREIGN KEY (id_instalacion) REFERENCES instalaciones(id_instalacion) ON DELETE CASCADE
 "fk_ofrecen_servicio" FOREIGN KEY (id_servicio) REFERENCES servicios(id_servicio) ON DELETE CASCADE

```

```

spa=> \d incluyen
          Tabla «public.incluyen»
  Columna |      Tipo      | Ordenamiento | Nulable | Por omisión
-----+-----+-----+-----+-----
 id_paquete | character(8) |              | not null | 
 id_servicio | character(8) |              | not null | 
Índices:
 "pk_incluyen" PRIMARY KEY, btree (id_paquete, id_servicio)
Restricciones de llave foránea:
 "fk_incluyen_paquetes" FOREIGN KEY (id_paquete) REFERENCES paquetes(id_paquete)
 "fk_incluyen_servicio" FOREIGN KEY (id_servicio) REFERENCES servicios(id_servicio)
ON DELETE RESTRICT
 "fk_incluyen_servicios" FOREIGN KEY (id_servicio) REFERENCES servicios(id_servicio)

```

```

[spa=> \d realizan
      Tabla «public.realizan»
      Columna | Tipo | Ordenamiento | Nulable | Por omisión
      -----+-----+-----+-----+-----
      id_servicio | character(8) | | not null |
      num_empleado | character(8) | | not null |
      fecha_r | date | | not null | CURRENT_DATE
Índices:
      "pk_realizan" PRIMARY KEY, btree (id_servicio, num_empleado, fecha_r)
      "idx_realizan_empleado" btree (num_empleado, fecha_r)
      "idx_realizan_servicio_fecha" btree (id_servicio, fecha_r, num_empleado)
Restricciones de llave foránea:
      "fk_realizan_empleado" FOREIGN KEY (num_empleado) REFERENCES empleados(num_empleado) ON DELETE CASCADE
      "fk_realizan_servicio" FOREIGN KEY (id_servicio) REFERENCES servicios(id_servicio)

```

Alta de cuando menos 10 registros a las tablas catálogo de la BD y los registros correspondientes en las tablas de relación para denotar las asociaciones 1:1, 1:M, M:N.

NOTA: Para generar los registros usamos LLMs y verificamos la coherencia con el modelo de negocio.

```
spa=> INSERT INTO Clientes (num_cliente, nombres_c, apellido_paterno_c, apellido_materno_c) VALUES
('CL000001','Ana María','López','García'),
('CL000002','Bruno','Hernández','Mora'),
('CL000003','Carla','Ramírez','Soto'),
('CL000004','Diego','Núñez','Rojas'),
('CL000005','Elena','Vega','Paz'),
('CL000006','Fabio','Cano','Ibarra'),
('CL000007','Gina','Salas','Reyes'),
('CL000008','Hugo','Cruz','Delgado'),
('CL000009','Ivana','Ortega','León'),
(['CL000010','Jorge','Pineda','Ruiz']);
INSERT 0 10
```

```
spa=> INSERT INTO Clientes (num_cliente, nombres_c, apellido_paterno_c, apellido_materno_c) VALUES
('CL000011','Laura','Méndez','Castillo'),
('CL000012','Miguel','Ríos','Guerrero'),
('CL000013','Natalia','Vargas','Silva'),
('CL000014','Óscar','Delgado','Peña'),
('CL000015','Patricia','Castro','Ríos');
INSERT 0 5
```

```
spa=> INSERT INTO Paquetes (id_paquete, nombre_paquete) VALUES
('PK000001','Básico'),
('PK000002','Relax'),
('PK000003','Premium'),
('PK000004','Novias'),
('PK000005','Detox'),
('PK000006','Hidratación'),
('PK000007','Express'),
('PK000008','Gold'),
('PK000009','Antiestrés'),
(['PK000010','Full Day']);
INSERT 0 10
```

```
spa=> INSERT INTO Servicios (id_servicio, nombre_servicio) VALUES
('SV000001','Masaje Relajante'),
('SV000002','Facial Hidratante'),
('SV000003','Manicure'),
('SV000004','Pedicure'),
('SV000005','Sauna'),
('SV000006','Baño Solar'),
('SV000007','Depilación'),
('SV000008','Exfoliación'),
('SV000009','Aromaterapia'),
(['SV000010','Reflexología');
INSERT 0 10
```



```
spa=> INSERT INTO Empleados (num_empleado, nombres_e, apellido_paterno_e, apellido_materno_e, categoria_e) VALUES
('EM000001','María','Pérez','Soto','Esteticista'),
('EM000002','Luis','Gómez','Rico','Masajista'),
('EM000003','Sofía','Campos','Luna','Recepcionista'),
('EM000004','Ricardo','Díaz','Neri','Gerente'),
('EM000005','Valeria','Beltrán','Iglesias','Esteticista'),
('EM000006','Tomás','Quintero','Solís','Masajista'),
('EM000007','Daniela','Rivas','Toledo','Esteticista'),
('EM000008','Pablo','Acosta','Serna','Terapeuta'),
('EM000009','Nora','Flores','Vite','Recepcionista'),
('EM000010','Ángel','Santos','Cuevas','Masajista');
INSERT 0 10
```

```
spa=> INSERT INTO Camas_Solares (num_serie, num_focos) VALUES
('CS000001',24),
('CS000002',28),
('CS000003',32),
('CS000004',36),
('CS000005',40),
('CS000006',24),
('CS000007',28),
('CS000008',32),
('CS000009',36),
('CS000010',40);
INSERT 0 10
```

```
spa=> INSERT INTO Instalaciones (id_instalacion, num_camas) VALUES
('IN000001',8),
('IN000002',10),
('IN000003',6),
('IN000004',12),
('IN000005',9),
('IN000006',7),
('IN000007',11),
('IN000008',10),
('IN000009',8),
('IN000010',14);
INSERT 0 10
```

```
spa=> INSERT INTO Contratan (num_cliente, id_paquete, fecha_c, metodo_pago) VALUES
('CL000001','PK000001','2025-05-01','TarjetaCrédito'),
('CL000002','PK000002','2025-05-02','Efectivo'),
('CL000003','PK000003','2025-05-03','Transferencia'),
('CL000004','PK000004','2025-05-04','TarjetaDébito'),
('CL000005','PK000005','2025-05-05','Efectivo'),
('CL000006','PK000006','2025-05-06','Depósito'),
('CL000007','PK000007','2025-05-07','Transferencia'),
('CL000008','PK000008','2025-05-08','TarjetaDébito'),
('CL000009','PK000009','2025-05-09','Efectivo'),
('CL000010','PK000010','2025-05-10','Transferencia');
INSERT 0 10
```

```
spa=> INSERT INTO Contratan (num_cliente, id_paquete, fecha_c, metodo_pago) VALUES('CL000001','PK000003','2025-05-15','Tarje
taCrédito'),
('CL000002','PK000003','2025-05-16','Efectivo'),
('CL000003','PK000003','2025-05-17','Transferencia'),
('CL000011','PK000003','2025-05-18','TarjetaDébito'),
('CL000012','PK000003','2025-05-19','Efectivo'),
('CL000004','PK000001','2025-05-20','Depósito'),
('CL000005','PK000001','2025-05-21','Transferencia'),
('CL000013','PK000004','2025-05-22','TarjetaCrédito'),
('CL000014','PK000004','2025-05-23','Efectivo'),
('CL000015','PK000002','2025-05-24','TarjetaDébito');
INSERT 0 10
```

```
spa=> INSERT INTO Ofrecen (id_servicio, num_serie, id_instalacion) VALUES
('SV000006','CS000001','IN000001'),
('SV000006','CS000002','IN000002'),
('SV000006','CS000003','IN000003'),
('SV000006','CS000004','IN000004'),
('SV000006','CS000005','IN000005'),
('SV000005','CS000006','IN000006'),
('SV000005','CS000007','IN000007'),
('SV000005','CS000008','IN000008'),
('SV000005','CS000009','IN000009'),
[('SV000005','CS000010','IN000010');
INSERT 0 10
```

```
spa=> INSERT INTO Incluyen (id_paquete, id_servicio) VALUES
('PK000001','SV000003'),
('PK000001','SV000004'),
('PK000002','SV000001'),
('PK000002','SV000009'),
('PK000003','SV000001'),
('PK000003','SV000002'),
('PK000004','SV000006'),
('PK000005','SV000007'),
('PK000006','SV000002'),
[('PK000008','SV000010');
INSERT 0 10
```

```
spa=> INSERT INTO Realizan (id_servicio, num_empleado, fecha_r) VALUES
('SV000001','EM000002','2025-05-11'),
('SV000002','EM000005','2025-05-11'),
('SV000003','EM000007','2025-05-12'),
('SV000004','EM000007','2025-05-12'),
('SV000005','EM000010','2025-05-12'),
('SV000006','EM000008','2025-05-13'),
('SV000007','EM000001','2025-05-13'),
('SV000008','EM000001','2025-05-14'),
('SV000009','EM000002','2025-05-14'),
[('SV000010','EM000010','2025-05-15');
INSERT 0 10
```

```
spa=> INSERT INTO REALIZAN VALUES ('SV000006','EM000006','2024-04-08'),('SV000006','EM000008','2024-03-21');
INSERT 0 2
```

Verificar que todo se haya guardado correctamente:


```
spa=> SELECT * FROM CLIENTES;
```

num_cliente	nombres_c	apellido_paterno_c	apellido_materno_c
CL000001	Ana María	López	García
CL000002	Bruno	Hernández	Mora
CL000003	Carla	Ramírez	Soto
CL000004	Diego	Núñez	Rojas
CL000005	Elena	Vega	Paz
CL000006	Fabio	Cano	Ibarra
CL000007	Gina	Salas	Reyes
CL000008	Hugo	Cruz	Delgado
CL000009	Ivana	Ortega	León
CL000010	Jorge	Pineda	Ruiz
CL000011	Laura	Méndez	Castillo
CL000012	Miguel	Ríos	Guerrero
CL000013	Natalia	Vargas	Silva
CL000014	Óscar	Delgado	Peña
CL000015	Patricia	Castro	Ríos

(15 filas)

```
spa=> SELECT * FROM paquetes;
```

id_paquete	nombre_paquete
PK000001	Básico
PK000002	Relax
PK000003	Premium
PK000004	Novias
PK000005	Detox
PK000006	Hidratación
PK000007	Express
PK000008	Gold
PK000009	Antiestrés
PK000010	Full Day

(10 filas)

```
spa=> SELECT * FROM servicios;
```

id_servicio	nombre_servicio
SV000001	Masaje Relajante
SV000002	Facial Hidratante
SV000003	Manicure
SV000004	Pedicure
SV000005	Sauna
SV000006	Baño Solar
SV000007	Depilación
SV000008	Exfoliación
SV000009	Aromaterapia
SV000010	Reflexología

(10 filas)

```
[spa=> SELECT * FROM empleados;
num_empleado | nombres_e | apellido_paterno_e | apellido_materno_e | categoria_e
-----+-----+-----+-----+-----
EM000001    | María    | Pérez              | Soto                | Esteticista
EM000002    | Luis     | Gómez              | Rico                | Masajista
EM000003    | Sofía    | Campos              | Luna                | Recepcionista
EM000004    | Ricardo  | Díaz               | Neri                | Gerente
EM000005    | Valeria  | Beltrán            | Iglesias            | Esteticista
EM000006    | Tomás    | Quintero           | Solís               | Masajista
EM000007    | Daniela  | Rivas              | Toledo              | Esteticista
EM000008    | Pablo    | Acosta             | Serna               | Terapeuta
EM000009    | Nora     | Flores             | Vite                | Recepcionista
EM000010    | Ángel    | Santos             | Cuevas              | Masajista
(10 filas)
```

```
[spa=> SELECT * FROM camas_solares;
num_serie | num_focos
-----+-----
CS000001  | 24
CS000002  | 28
CS000003  | 32
CS000004  | 36
CS000005  | 40
CS000006  | 24
CS000007  | 28
CS000008  | 32
CS000009  | 36
CS000010  | 40
(10 filas)
```

```
[spa=> SELECT * FROM instalaciones;
id_instalacion | num_camas
-----+-----
IN000001       | 8
IN000002       | 10
IN000003       | 6
IN000004       | 12
IN000005       | 9
IN000006       | 7
IN000007       | 11
IN000008       | 10
IN000009       | 8
IN000010       | 14
(10 filas)
```

```
[spa=> SELECT * FROM CONTRATAN;
num_cliente | id_paquete | fecha_c | metodo_pago
-----+-----+-----+-----
CL000001    | PK000001   | 2025-05-01 | TarjetaCrédito
CL000002    | PK000002   | 2025-05-02 | Efectivo
CL000003    | PK000003   | 2025-05-03 | Transferencia
CL000004    | PK000004   | 2025-05-04 | TarjetaDébito
CL000005    | PK000005   | 2025-05-05 | Efectivo
CL000006    | PK000006   | 2025-05-06 | Depósito
CL000007    | PK000007   | 2025-05-07 | Transferencia
CL000008    | PK000008   | 2025-05-08 | TarjetaDébito
CL000009    | PK000009   | 2025-05-09 | Efectivo
CL000010    | PK000010   | 2025-05-10 | Transferencia
CL000001    | PK000003   | 2025-05-15 | TarjetaCrédito
CL000002    | PK000003   | 2025-05-16 | Efectivo
CL000003    | PK000003   | 2025-05-17 | Transferencia
CL000011    | PK000003   | 2025-05-18 | TarjetaDébito
CL000012    | PK000003   | 2025-05-19 | Efectivo
CL000004    | PK000001   | 2025-05-20 | Depósito
CL000005    | PK000001   | 2025-05-21 | Transferencia
CL000013    | PK000004   | 2025-05-22 | TarjetaCrédito
CL000014    | PK000004   | 2025-05-23 | Efectivo
CL000015    | PK000002   | 2025-05-24 | TarjetaDébito
(20 filas)
```

```
[spa=> SELECT * FROM ofrecen;
id_servicio | num_serie | id_instalacion
-----+-----+-----
SV000006    | CS000001   | IN000001
SV000006    | CS000002   | IN000002
SV000006    | CS000003   | IN000003
SV000006    | CS000004   | IN000004
SV000006    | CS000005   | IN000005
SV000005    | CS000006   | IN000006
SV000005    | CS000007   | IN000007
SV000005    | CS000008   | IN000008
SV000005    | CS000009   | IN000009
SV000005    | CS000010   | IN000010
(10 filas)
```



```
[spa=> SELECT * FROM incluyen;
id_paquete | id_servicio
-----+-----
PK000001   | SV000003
PK000001   | SV000004
PK000002   | SV000001
PK000002   | SV000009
PK000003   | SV000001
PK000003   | SV000002
PK000004   | SV000006
PK000005   | SV000007
PK000006   | SV000002
PK000008   | SV000010
(10 filas)
```

```
[spa=> SELECT * FROM REALIZAN;
id_servicio | num_empleado | fecha_r
-----+-----+-----
SV000001   | EM000002     | 2025-05-11
SV000002   | EM000005     | 2025-05-11
SV000003   | EM000007     | 2025-05-12
SV000004   | EM000007     | 2025-05-12
SV000005   | EM000010     | 2025-05-12
SV000006   | EM000008     | 2025-05-13
SV000007   | EM000001     | 2025-05-13
SV000008   | EM000001     | 2025-05-14
SV000009   | EM000002     | 2025-05-14
SV000010   | EM000010     | 2025-05-15
SV000006   | EM000006     | 2024-04-08
SV000006   | EM000008     | 2024-03-21
(12 filas)
```

Creación de índices en campos líderes de consulta común.

```
-- Conectarse a la base de datos
\c spa;

-- Índice para búsqueda por nombre de empleado
CREATE UNIQUE INDEX idx_empleados_nombre_unique ON Empleados (nombres_e, apellido_paterno_e,
apellido_materno_e);

-- Índice para relacionar empleados con servicios realizados
CREATE INDEX idx_realizan_empleado ON Realizan (num_empleado, fecha_r);

-- Índice para contar sesiones por servicio (usando Realizan)
CREATE INDEX idx_realizan_servicio_fecha ON Realizan (id_servicio, fecha_r, num_empleado);

-- Índice para búsqueda por nombre de paquete
CREATE INDEX idx_paquetes_nombre ON Paquetes (nombre_paquete, id_paquete);

-- Índice para nombres de clientes (si necesitas ordenar o buscar por nombre)
CREATE UNIQUE INDEX idx_clientes_nombre_unique ON Clientes (nombres_c, apellido_paterno_c,
apellido_materno_c);
```

```
spa=> -- Índice para búsqueda por nombre de empleado
CREATE UNIQUE INDEX idx_empleados_nombre_unique ON Empleados (nombres_e, apellido_paterno_e, apellido_materno_e);

-- Índice para relacionar empleados con servicios realizados
CREATE INDEX idx_realizan_empleado ON Realizan (num_empleado, fecha_r);

-- Índice para contar sesiones por servicio (usando Realizan)
CREATE INDEX idx_realizan_servicio_fecha ON Realizan (id_servicio, fecha_r, num_empleado);

-- Índice para búsqueda por nombre de paquete
CREATE INDEX idx_paquetes_nombre ON Paquetes (nombre_paquete, id_paquete);

-- Índice para nombres de clientes (si necesitas ordenar o buscar por nombre)
CREATE UNIQUE INDEX idx_clientes_nombre_unique ON Clientes (nombres_c, apellido_paterno_c, apellido_materno_c);
CREATE INDEX
CREATE INDEX
CREATE INDEX
CREATE INDEX
CREATE INDEX
```

Procedimiento almacenado sp_inserta que, a través de parámetros permita la inserción de registros de alguna tabla catálogo.

Nuestras tablas catálogo son: clientes, paquetes, servicios, empleados, camas_solares e instalaciones. Como cada una tiene diferentes atributos, para dar de alta un registro en cada una debemos crear una función 'sp_inserta' para cada nuevo registro.

```
spa=> CREATE OR REPLACE PROCEDURE sp_inserta_cliente(
    num_cliente_p CHAR(8),
    nombres_c_p VARCHAR(30),
    apellido_paterno_c_p VARCHAR(30),
    apellido_materno_c_p VARCHAR(30))
LANGUAGE plpgsql
AS $$
BEGIN
    -- checamos constraints
    -- verificamos que los parámetros sean correctos y esten completos
    IF num_cliente_p IS NULL OR nombres_c_p IS NULL OR apellido_paterno_c_p IS NULL THEN
        RAISE EXCEPTION 'num_cliente, nombres_c y apellido_paterno_c son parametros obligatorios';
    END IF;

    IF nombres_c_p !~ '^[A-ZÁÉÍÓÚÑa-záéíóúñ\s]+$' THEN
        RAISE EXCEPTION 'El nombre solo puede contener letras y espacios: %', nombres_c_p;
    END IF;

    IF apellido_paterno_c_p !~ '^[A-ZÁÉÍÓÚÑa-záéíóúñ]+$' THEN
        RAISE EXCEPTION 'El apellido paterno solo puede contener letras: %', apellido_paterno_c_p;
    END IF;

    -- si todo bien insertamos
    INSERT INTO clientes (num_cliente, nombres_c, apellido_paterno_c, apellido_materno_c)
    VALUES(num_cliente_p, nombres_c_p, apellido_paterno_c_p, apellido_materno_c_p);

    -- avisamos que lo hicimos correctamente
    RAISE NOTICE 'Cliente % insertado correctamente', num_cliente_p;

    -- cachamos excepciones por si ya existe el num_cliente
    EXCEPTION
        WHEN unique_violation THEN
            RAISE EXCEPTION 'El cliente % ya existe', num_cliente_p;
END; $$;
CREATE PROCEDURE
```

```
spa=> CREATE OR REPLACE PROCEDURE sp_inserta_paquetes(
    id_paquete_p CHAR(8),
    nombre_paquete_p VARCHAR(30))
LANGUAGE plpgsql
AS $$
BEGIN
    -- checamos constraints
    IF nombre_paquete_p IS NULL THEN
        RAISE EXCEPTION 'nombre_paquete es un parametro obligatorio';
    END IF;

    -- si todo bien insertamos
    INSERT INTO paquetes (id_paquete, nombre_paquete)
    VALUES(id_paquete_p, nombre_paquete_p);

    -- avisamos que lo hicimos correctamente
    RAISE NOTICE 'Paquete % insertado correctamente', id_paquete_p;

    -- cachamos excepciones por si ya existe el id_paquete
    EXCEPTION
        WHEN unique_violation THEN
            RAISE EXCEPTION 'El Paquete % ya existe', id_paquete_p;
END; $$;
CREATE PROCEDURE
```



```

spa=> CREATE OR REPLACE PROCEDURE sp_inserta_servicios(
        id_servicio_p CHAR(8),
        nombre_servicio_p VARCHAR(30))
LANGUAGE plpgsql
AS $$
BEGIN
    -- checamos constraints
    IF nombre_servicio_p IS NULL THEN
        RAISE EXCEPTION 'nombre_servicio es un parametro obligatorio';
    END IF;

    -- si todo bien insertamos
    INSERT INTO servicios(id_servicio, nombre_servicio)
    VALUES(id_servicio_p, nombre_servicio_p);

    -- avisamos que lo hicimos correctamente
    RAISE NOTICE 'Servicio % insertado correctamente', id_servicio_p;

    -- cachamos excepciones por si ya existe el id_servicio
EXCEPTION
    WHEN unique_violation THEN
        RAISE EXCEPTION 'El Servicio % ya existe', id_servicio_p;
END; $$;
CREATE PROCEDURE

```

```

spa=> CREATE OR REPLACE PROCEDURE sp_inserta_empleados(
        num_empleado_p CHAR(8),
        nombres_e_p VARCHAR(30),
        apellido_paterno_e_p VARCHAR(30),
        apellido_materno_e_p VARCHAR(30),
        categoria_e_p VARCHAR(30))
LANGUAGE plpgsql
AS $$
BEGIN
    -- checamos constraints
    IF num_empleado_p IS NULL OR nombres_e_p IS NULL OR apellido_paterno_e_p IS NULL OR categoria_e_p IS NULL THEN
        RAISE EXCEPTION 'num_empleado, nombres_e, apellido_paterno_e y categoria_e son parametros obligatorios';
    END IF;

    IF categoria_e_p NOT IN ('Recepcionista', 'Terapeuta', 'Masajista', 'Esteticista', 'Gerente') THEN RAISE EXCEPTION 'Categoria de empleado no valida: %. Las categorías permitidas son: Recepcionista, Terapeuta, Masajista, Esteticista, Gerente', categoria_e_p;
    END IF;

    IF nombres_e_p !~ '^[A-ZÁÉÍÓÚÑá-záéíóúñ\s]+$' THEN
        RAISE EXCEPTION 'El nombre solo puede contener letras y espacios: %', nombres_e_p;
    END IF;

    IF apellido_paterno_e_p !~ '^[A-ZÁÉÍÓÚÑá-záéíóúñ]+$' THEN
        RAISE EXCEPTION 'El apellido paterno solo puede contener letras: %', apellido_paterno_e_p;
    END IF;

    -- si todo bien insertamos
    INSERT INTO empleados(num_empleado, nombres_e, apellido_paterno_e, apellido_materno_e, categoria_e)
    VALUES(num_empleado_p, nombres_e_p, apellido_paterno_e_p, apellido_materno_e_p, categoria_e_p);

    -- avisamos que lo hicimos correctamente
    RAISE NOTICE 'Empleado % insertado correctamente', num_empleado_p;

    -- cachamos excepciones por si ya existe el num_empleado_p
EXCEPTION
    WHEN unique_violation THEN
        RAISE EXCEPTION 'El Empleado % ya existe', num_empleado_p;
END; $$;
CREATE PROCEDURE

```



```

spa=> CREATE OR REPLACE PROCEDURE sp_inserta_camas_solares(
        num_serie_p CHAR(8),
        num_focos_p INTEGER)
LANGUAGE plpgsql
AS $$
BEGIN
    -- checamos constraints
    IF num_focos_p <= 0 THEN
        RAISE EXCEPTION 'El numero de focos debe ser positivo.';
    END IF;

    -- si todo bien insertamos
    INSERT INTO camas_solares (num_serie, num_focos)
    VALUES(num_serie_p, num_focos_p);

    -- avisamos que lo hicimos correctamente
    RAISE NOTICE 'Cama solar % insertada correctamente', num_serie_p;

-- cachamos excepciones por si ya existe el num_serie
EXCEPTION
    WHEN unique_violation THEN
        RAISE EXCEPTION 'La cama % ya existe', num_serie_p;
END; $$;
CREATE PROCEDURE

```

```

spa=> CREATE OR REPLACE PROCEDURE sp_inserta_instalaciones(
        id_instalacion_p CHAR(8),
        num_camas_p INTEGER)
LANGUAGE plpgsql
AS $$
BEGIN
    -- checamos constraints
    IF num_camas_p <= 0 THEN
        RAISE EXCEPTION 'El numero de camas solares debe ser positivo.';
    END IF;

    -- si todo bien insertamos
    INSERT INTO instalaciones(id_instalacion, num_camas)
    VALUES(id_instalacion_p, num_camas_p);

    -- avisamos que lo hicimos correctamente
    RAISE NOTICE 'Instalacion % insertada correctamente', id_instalacion_p;

-- cachamos excepciones por si ya existe el id_instalacion
EXCEPTION
    WHEN unique_violation THEN
        RAISE EXCEPTION 'La instalacion % ya existe', id_instalacion_p;
END; $$;

```

Probamos cada una de los procedimientos almacenados:

```

[spa=> CALL sp_inserta_cliente('CL000016', 'Sofía', 'Martínez', 'López');
NOTICE: Cliente CL000016 insertado correctamente
CALL

```

```

[spa=> CALL sp_inserta_cliente('CL000019', NULL, 'Hernández', 'Mora');
ERROR: num_cliente, nombres_c y apellido_paterno_c son parametros obligatorios
CONTEXTO: función PL/pgSQL sp_inserta_cliente(character,character varying,character varying,character varying) en la línea 6 en RAISE

```

```
spa=> SELECT*FROM CLIENTES;
```

num_cliente	nombres_c	apellido_paterno_c	apellido_materno_c
CL000001	Ana María	López	García
CL000002	Bruno	Hernández	Mora
CL000003	Carla	Ramírez	Soto
CL000004	Diego	Núñez	Rojas
CL000005	Elena	Vega	Paz
CL000006	Fabio	Cano	Ibarra
CL000007	Gina	Salas	Reyes
CL000008	Hugo	Cruz	Delgado
CL000009	Ivana	Ortega	León
CL000010	Jorge	Pineda	Ruiz
CL000011	Laura	Méndez	Castillo
CL000012	Miguel	Ríos	Guerrero
CL000013	Natalia	Vargas	Silva
CL000014	Óscar	Delgado	Peña
CL000015	Patricia	Castro	Ríos
CL000016	Sofía	Martínez	López

(16 filas)

```
[spa=> CALL sp_inserta_paquetes('PK000011', 'Spa Completo Premium');
NOTICE: Paquete PK000011 insertado correctamente
CALL
```

```
[spa=> CALL sp_inserta_paquetes('PK000012', NULL);
ERROR: nombre_paquete es un parametro obligatorio
CONTEXTO: función PL/pgSQL sp_inserta_paquetes(character,character varying) en la
línea 5 en RAISE
```

```
spa=> SELECT*FROM PAQUETES;
```

id_paquete	nombre_paquete
PK000001	Básico
PK000002	Relax
PK000003	Premium
PK000004	Novias
PK000005	Detox
PK000006	Hidratación
PK000007	Express
PK000008	Gold
PK000009	Antiestrés
PK000010	Full Day
PK000011	Spa Completo Premium

(11 filas)

```
[spa=> CALL sp_inserta_servicios('SV000011', 'Masaje con Piedras Calientes');
NOTICE: Servicio SV000011 insertado correctamente
CALL
```

```
spa=> CALL sp_inserta_servicios('SV000012', NULL);
ERROR: nombre_servicio es un parametro obligatorio
CONTEXTO: función PL/pgSQL sp_inserta_servicios(character,character varying) en l
a línea 5 en RAISE
```

```
[spa=> SELECT*FROM SERVICIOS;
id_servicio | nombre_servicio
-----+-----
SV000001    | Masaje Relajante
SV000002    | Facial Hidratante
SV000003    | Manicure
SV000004    | Pedicure
SV000005    | Sauna
SV000006    | Baño Solar
SV000007    | Depilación
SV000008    | Exfoliación
SV000009    | Aromaterapia
SV000010    | Reflexología
SV000011    | Masaje con Piedras Calientes
(11 filas)
```

```
[spa=> CALL sp_inserta_empleados('EM000011', 'Laura', 'Díaz', 'Romero', 'Esteticista');
NOTICE: Empleado EM000011 insertado correctamente
CALL
```

```
[spa=> CALL sp_inserta_empleados('EM000013', 'Ana123', 'Martínez', 'Sánchez', 'Recepcionista');
ERROR: El nombre solo puede contener letras y espacios: Ana123
CONTEXTO: función PL/pgSQL sp_inserta_empleados(character,character varying,character varying,character varying,character varying) en la línea 12 en RAISE
```

```
[spa=> SELECT*FROM EMPLEADOS;
num_empleado | nombres_e | apellido_paterno_e | apellido_materno_e | categoria_e
-----+-----+-----+-----+-----
EM000001     | María    | Pérez              | Soto                | Esteticista
EM000002     | Luis     | Gómez              | Rico                | Masajista
EM000003     | Sofía    | Campos             | Luna                | Recepcionista
EM000004     | Ricardo  | Díaz               | Neri                | Gerente
EM000005     | Valeria  | Beltrán            | Iglesias            | Esteticista
EM000006     | Tomás    | Quintero           | Solís               | Masajista
EM000007     | Daniela  | Rivas              | Toledo              | Esteticista
EM000008     | Pablo    | Acosta             | Serna               | Terapeuta
EM000009     | Nora     | Flores             | Vite                | Recepcionista
EM000010     | Ángel    | Santos             | Cuevas              | Masajista
EM000011     | Laura    | Díaz              | Romero              | Esteticista
(11 filas)
```

```
[spa=> CALL sp_inserta_camras_solares('CS000011', 42);
NOTICE: Cama solar CS000011 insertada correctamente
CALL
[spa=> CALL sp_inserta_camras_solares('CS000012', 0);
ERROR: El numero de focos debe ser positivo.
CONTEXTO: función PL/pgSQL sp_inserta_camras_solares(character,integer) en la línea 5 en RAISE
```



```
[spa=> SELECT*FROM CAMAS_SOLARES;
 num_serie | num_focos
-----+-----
CS000001  |         24
CS000002  |         28
CS000003  |         32
CS000004  |         36
CS000005  |         40
CS000006  |         24
CS000007  |         28
CS000008  |         32
CS000009  |         36
CS000010  |         40
CS000011  |         42
(11 filas)
```

```
[spa=> CALL sp_inserta_instalaciones('IN000011', 15);
NOTICE: Instalacion IN000011 insertada correctamente
CALL
spa=> CALL sp_inserta_instalaciones('IN000013', -3);
ERROR: El numero de camas solares debe ser positivo.
CONTEXT0: función PL/pgSQL sp_inserta_instalaciones(character,integer) en la línea
a 5 en RAISE
```

```
[spa=> SELECT*FROM INSTALACIONES;
 id_instalacion | num_camas
-----+-----
IN000001        |         8
IN000002        |        10
IN000003        |         6
IN000004        |        12
IN000005        |         9
IN000006        |         7
IN000007        |        11
IN000008        |        10
IN000009        |         8
IN000010        |        14
IN000011        |        15
(11 filas)
```


Creación de consultas, que satisfagan la hoja de requerimientos.

i) Dado el nombre de un empleado, conocer las sesiones que ha aplicado y los clientes que atendió en cada sesión.

```
spa=>
SELECT s.nombre_servicio, r.fecha_r AS fecha_sesion,
c.nombres_c || ' ' || c.apellido_paterno_c || ' ' || c.apellido_materno_c AS nombre_cliente
FROM Empleados e
INNER JOIN Realizan r ON e.num_empleado = r.num_empleado
INNER JOIN Servicios s ON r.id_servicio = s.id_servicio
INNER JOIN Incluyen i ON s.id_servicio = i.id_servicio
INNER JOIN Paquetes p ON i.id_paquete = p.id_paquete
INNER JOIN Contratan ct ON p.id_paquete = ct.id_paquete
INNER JOIN Clientes c ON ct.num_cliente = c.num_cliente
WHERE e.nombres_e = 'Pablo' AND e.apellido_paterno_e = 'Acosta'
ORDER BY r.fecha_r DESC;
```

nombre_servicio	fecha_sesion	nombre_cliente
Baño Solar	2025-05-13	Diego Núñez Rojas
Baño Solar	2025-05-13	Natalia Vargas Silva
Baño Solar	2025-05-13	Óscar Delgado Peña
Baño Solar	2024-03-21	Diego Núñez Rojas
Baño Solar	2024-03-21	Natalia Vargas Silva
Baño Solar	2024-03-21	Óscar Delgado Peña

(6 filas)

ii) Dado el nombre de una cama solar, conocer el nombre de los empleados que la usan con frecuencia y el número de sesiones que han realizado en ella.

```
spa=> SELECT e.nombres_e || ' ' || e.apellido_paterno_e || ' ' || e.apellido_materno_e AS nombre_empleado,
COUNT(r.id_servicio) AS total_sesiones, -- Cuántas veces ha usado esta cama el empleado
MIN(r.fecha_r) AS primera_sesion, -- Primera vez que usó la cama
MAX(r.fecha_r) AS ultima_sesion -- Última vez que usó la cama
FROM Camas_Solares cs
INNER JOIN Ofrecen o ON cs.num_serie = o.num_serie -- Une camas con servicios que ofrecen
INNER JOIN Servicios s ON o.id_servicio = s.id_servicio -- Une con nombres de servicios
INNER JOIN Realizan r ON s.id_servicio = r.id_servicio -- Une servicios con empleados que los realizan
INNER JOIN Empleados e ON r.num_empleado = e.num_empleado -- Une con datos de empleados
WHERE cs.num_serie = 'CS000005' -- Buscamos la cama con número de serie CS000005
GROUP BY cs.num_serie, e.num_empleado, e.nombres_e, e.apellido_paterno_e, e.apellido_materno_e, s.nombre_servicio
ORDER BY total_sesiones DESC; -- Ordenamos por quien la usa mas.
```

nombre_empleado	total_sesiones	primera_sesion	ultima_sesion
Pablo Acosta Serna	2	2024-03-21	2025-05-13
Tomás Quintero Solís	1	2024-04-08	2024-04-08

(2 filas)

iii) Dado el nombre de un paquete, conocer el nombre de aquellos clientes que lo han contratado y el método de pago que han usado.

```
spa=> SELECT c.nombres_c || ' ' || c.apellido_paterno_c || ' ' || c.apellido_materno_c AS nombre_cliente_completo, ct.fecha_
c AS fecha_contratacion, ct.metodo_pago
FROM Paquetes p
INNER JOIN Contratan ct ON p.id_paquete = ct.id_paquete          -- Une paquetes con clientes que los compraron
INNER JOIN Clientes c ON ct.num_cliente = c.num_cliente          -- Une con datos de clientes
WHERE p.nombre_paquete = 'Premium'                                -- Buscamos el paquete 'Premium'
GROUP BY p.nombre_paquete, c.num_cliente, c.nombres_c, c.apellido_paterno_c, c.apellido_materno_c, ct.fecha_c, ct.metodo_pag
o
ORDER BY ct.fecha_c DESC;
]
nombre_cliente_completo | fecha_contratacion | metodo_pago
-----
Miguel Ríos Guerrero    | 2025-05-19         | Efectivo
Laura Méndez Castillo   | 2025-05-18         | TarjetaDébito
Carla Ramírez Soto      | 2025-05-17         | Transferencia
Bruno Hernández Mora    | 2025-05-16         | Efectivo
Ana María López García  | 2025-05-15         | TarjetaCrédito
Carla Ramírez Soto      | 2025-05-03         | Transferencia
(6 filas)
```

Procedimiento almacenado `sp_consultas` que acepte parámetros y ejecute 5 consultas, si los requerimientos contienen menos crear dos nuevas. La salida del procedimiento (es decir, el resultado de las consultas) debe guardarlo en un archivo.

Revisamos qué procedimientos están disponibles:

```
spa=> \df
```

Esquema	Nombre	Tipo de dato de salida	Tipo	Listado de funciones	Tipos de datos de argumentos
public	sp_borra			p_tabla character varying, p_id_registro character varying	
public	sp_inserta_camras_solares			num_serie_p character, num_focos_p integer	
public	sp_inserta_cliente			num_cliente_p character, nombres_c_p character varying, apellido_paterno_c_p character varying	
public	sp_inserta_empleados			num_empleado_p character, nombres_e_p character varying, apellido_paterno_e_p character varying	
public	sp_inserta_instalaciones			id_instalacion_p character, num_camras_p integer	
public	sp_inserta_paquetes			id_paquete_p character, nombre_paquete_p character varying	
public	sp_inserta_servicios			id_servicio_p character, nombre_servicio_p character varying	

(7 filas)

Creamos el procedimiento (el script del procedimiento se adjunta en la entrega):

```
spa=> CREATE OR REPLACE PROCEDURE sp_consultas(
    empleado_nombre CHARACTER VARYING,
    empleado_apellido CHARACTER VARYING,
    cama_serie CHARACTER VARYING,
    paquete_nombre CHARACTER VARYING,
    fecha_desde DATE,
    fecha_hasta DATE,
    instalacion_id CHARACTER VARYING,
    archivo_salida TEXT DEFAULT '/home/alumno06/reporte_spa.txt'
)
LANGUAGE plpgsql
AS $$
BEGIN
    -- Crear tabla temporal para almacenar el reporte
    DROP TABLE IF EXISTS tmp_consultas_export;
    CREATE TEMP TABLE tmp_consultas_export (
        line TEXT
    );
    -- a veces nos lanza 'la tabla «tmp_consultas_export» no existe, omitiendo' si ejecutamos el call en otra linea pero si funciona todo bien asi que podemos omitirlo.

    INSERT INTO tmp_consultas_export(line)
    WITH
    -- CONSULTA 1. Dado el nombre de un empleado, conocer las sesiones que ha aplicado y los clientes que atendio en cada sesion

    c1_rows AS (
        SELECT
            s.nombre_servicio,
            r.fecha_r AS fecha_sesion,
            c.nombres_c || ' ' || c.apellido_paterno_c || ' ' || c.apellido_materno_c AS nombre_cliente
        FROM Empleados e
        INNER JOIN Realizan r ON e.num_empleado = r.num_empleado
        INNER JOIN Servicios s ON r.id_servicio = s.id_servicio
        INNER JOIN Incluyen i ON s.id_servicio = i.id_servicio
        INNER JOIN Paquetes p ON i.id_paquete = p.id_paquete
        INNER JOIN Contratan ct ON p.id_paquete = ct.id_paquete
        INNER JOIN Clientes c ON ct.num_cliente = c.num_cliente
        WHERE e.nombres_e = empleado_nombre AND e.apellido_paterno_e = empleado_apellido
        ORDER BY r.fecha_r DESC
    ),
    $$;RAISE NOTICE 'Para exportar a TXT: \copy (SELECT * FROM tmp_consultas_export) TO '%';', archivo_salida;=:TEXTodo de pago que han usadohan real
CREATE PROCEDURE
```

Checamos que sí exista en los procedimientos disponibles:

```
spa=> \df
```

Esquema	Nombre	Tipo de dato de salida	Tipo	Listado de funciones	Tipos de datos de argumentos
public	sp_borra			p_tabla character varying, p_id_registro character varying	
public	sp_consultas			empleado_nombre character varying, empleado_apellido character varying, cama_serie character varying, paquete_nombre character varying, fecha_desde date, fecha_hasta date, instalacion_id character varying, archivo_salida text DEFAULT '/home/alumno06/reporte_spa.txt':text	
public	sp_inserta_camras_solares			num_serie_p character, num_focos_p integer	
public	sp_inserta_cliente			num_cliente_p character, nombres_c_p character varying, apellido_paterno_c_p character varying, apellido_materno_c_p character varying	
public	sp_inserta_empleados			num_empleado_p character, nombres_e_p character varying, apellido_paterno_e_p character varying, apellido_materno_e_p character varying	
public	sp_inserta_instalaciones			id_instalacion_p character, num_camras_p integer	
public	sp_inserta_paquetes			id_paquete_p character, nombre_paquete_p character varying	
public	sp_inserta_servicios			id_servicio_p character, nombre_servicio_p character varying	

(8 filas)

Lo probamos:

```
spa=> CALL sp_consultas(
    'Pablo',
    'Acosta',
    'CS000005',
    'Premium',
    '2024-01-01',
    '2024-12-31',
    'IN000004',
    '/home/alumno06/mi_reporte.txt'
);
NOTICE: Consultas ejecutadas exitosamente!
NOTICE: Para ver los resultados ejecuta: SELECT * FROM tmp_consultas_export;
NOTICE: Para exportar a TXT: \copy (SELECT * FROM tmp_consultas_export) TO '/home/alumno06/mi_reporte.txt';
CALL
```

Los visualizamos:

```
spa=> SELECT * FROM tmp_consultas_export;
line

==== CONSULTA 1: Sesiones del empleado Pablo Acosta ====
Servicio | Fecha Sesión | Cliente
-----
Baño Solar | 2025-05-13 | Diego Núñez Rojas
Baño Solar | 2025-05-13 | Natalia Vargas Silva
Baño Solar | 2025-05-13 | Óscar Delgado Peña
Baño Solar | 2024-03-21 | Diego Núñez Rojas
Baño Solar | 2024-03-21 | Natalia Vargas Silva
Baño Solar | 2024-03-21 | Óscar Delgado Peña

==== CONSULTA 2: Empleados que usan la cama CS000005 ====
Empleado | Total Sesiones | Primera Sesión | Última Sesión
-----
Pablo Acosta Serna | 2 | 2024-03-21 | 2025-05-13
Tomás Quintero Solís | 1 | 2024-04-08 | 2024-04-08

==== CONSULTA 3: Clientes que contrataron el paquete Premium ====
Cliente | Fecha Contratación | Método de Pago
-----
Miguel Ríos Guerrero | 2025-05-19 | Efectivo
Laura Méndez Castillo | 2025-05-18 | TarjetaDébito
Carla Ramírez Soto | 2025-05-17 | Transferencia
Bruno Hernández Mora | 2025-05-16 | Efectivo
Carla Ramírez Soto | 2025-05-03 | Transferencia

==== CONSULTA 4: Servicios más utilizados del 2024-01-01 al 2024-12-31 ====
Servicio | Total Sesiones | Empleados | Última Sesión
-----
Baño Solar | 2 | 2 | 2024-04-08

==== CONSULTA 5: Empleados frecuentes en instalación IN000004 ====
Empleado | Categoría | Total Servicios | Última Actividad
-----
Pablo Acosta Serna | Terapeuta | 2 | 2025-05-13
Tomás Quintero Solís | Masajista | 1 | 2024-04-08

==== FIN DEL REPORTE ====
(37 filas)
```

NOTA IMPORTANTE: Como no tenemos permisos para copiar porque no somos administradores de la base de datos, por eso el procedimiento como tal no puede generar el archivo, debemos hacer una copia posterior al procedimiento, con: **\copy (SELECT * FROM tmp_consultas_export) TO '/home/alumno06/mi_reporte.txt';**.

```
[spa=> \copy (SELECT * FROM tmp_consultas_export) TO '/home/alumno06/mi_reporte.txt';
COPY 37
```

Vemos el contenido del archivo:


```

[alumno06@master ~]$ cat /home/alumno06/mi_reporte.txt
=== CONSULTA 1: Sesiones del empleado Pablo Acosta ===
Servicio | Fecha Sesión | Cliente
-----
Baño Solar | 2025-05-13 | Diego Núñez Rojas
Baño Solar | 2025-05-13 | Natalia Vargas Silva
Baño Solar | 2025-05-13 | Óscar Delgado Peña
Baño Solar | 2024-03-21 | Diego Núñez Rojas
Baño Solar | 2024-03-21 | Natalia Vargas Silva
Baño Solar | 2024-03-21 | Óscar Delgado Peña

=== CONSULTA 2: Empleados que usan la cama CS000005 ===
Empleado | Total Sesiones | Primera Sesión | Última Sesión
-----
Pablo Acosta Serna | 2 | 2024-03-21 | 2025-05-13
Tomás Quintero Solís | 1 | 2024-04-08 | 2024-04-08

=== CONSULTA 3: Clientes que contrataron el paquete Premium ===
Cliente | Fecha Contratación | Método de Pago
-----
Miguel Ríos Guerrero | 2025-05-19 | Efectivo
Laura Méndez Castillo | 2025-05-18 | TarjetaDébito
Carla Ramírez Soto | 2025-05-17 | Transferencia
Bruno Hernández Mora | 2025-05-16 | Efectivo
Carla Ramírez Soto | 2025-05-03 | Transferencia

=== CONSULTA 4: Servicios más utilizados del 2024-01-01 al 2024-12-31 ===
Servicio | Total Sesiones | Empleados | Última Sesión
-----
Baño Solar | 2 | 2 | 2024-04-08

=== CONSULTA 5: Empleados frecuentes en instalación IN000004 ===
Empleado | Categoría | Total Servicios | Última Actividad
-----
Pablo Acosta Serna | Terapeuta | 2 | 2025-05-13
Tomás Quintero Solís | Masajista | 1 | 2024-04-08

=== FIN DEL REPORTE ===
[alumno06@master ~]$ █

```

Procedimiento almacenado sp_borra que permite eliminación de registros de alguna tabla catálogo.

```
spa=> CREATE OR REPLACE PROCEDURE sp_borra(
    p_tabla VARCHAR(50),
    p_id_registro VARCHAR(8))
LANGUAGE plpgsql
AS $$
DECLARE
    cuenta_registros_eliminados INT;
BEGIN
    -- Validamos tabla
    IF p_tabla NOT IN ('clientes', 'empleados', 'paquetes', 'servicios', 'camas_solares', 'instalaciones') THEN
        RAISE EXCEPTION 'Esa tabla no se puede usar, intenta con: clientes, empleados, paquetes, servicios, camas_solares o instalaciones';
    END IF;

    -- Determinamos que eliminar dependiendo del nombre de la tabla
    CASE p_tabla
        WHEN 'clientes' THEN
            DELETE FROM clientes WHERE num_cliente = p_id_registro;
        WHEN 'empleados' THEN
            DELETE FROM empleados WHERE num_empleado = p_id_registro;
        WHEN 'paquetes' THEN
            DELETE FROM paquetes WHERE id_paquete = p_id_registro;
        WHEN 'servicios' THEN
            DELETE FROM servicios WHERE id_servicio = p_id_registro;
        WHEN 'camas_solares' THEN
            DELETE FROM camas_solares WHERE num_serie = p_id_registro;
        WHEN 'instalaciones' THEN
            DELETE FROM instalaciones WHERE id_instalacion = p_id_registro;
    END CASE;

    -- usamos GET DIAGNOSIS para saber cuantos registros si se eliminaron
    GET DIAGNOSTICS cuenta_registros_eliminados = ROW_COUNT;

    IF cuenta_registros_eliminados = 0 THEN
        RAISE NOTICE 'Registro: % no se pudo eliminar porque no se encontró en la tabla %', p_id_registro, p_tabla;
    ELSE
        RAISE NOTICE 'Registro: % eliminado de la tabla %', p_id_registro, p_tabla;
    END IF;

    -- cachamos excepciones por constraint
    EXCEPTION
        WHEN foreign_key_violation THEN
            RAISE EXCEPTION 'No se puede eliminar. El registro tiene relaciones en otras tablas.';
END;
$$;
CREATE PROCEDURE
```

Probamos el procedimiento almacenado:

```
spa=> CALL sp_borra('clientes', 'CL000001'); -- SI FUNCIONA
NOTICE: Registro: CL000001 eliminado de la tabla clientes
CALL
```

```
[spa=> SELECT * FROM CLIENTES;
num_cliente | nombres_c | apellido_paterno_c | apellido_materno_c
-----+-----+-----+-----
CL000002    | Bruno    | Hernández         | Mora
CL000003    | Carla    | Ramírez           | Soto
CL000004    | Diego    | Núñez             | Rojas
CL000005    | Elena    | Vega              | Paz
CL000006    | Fabio    | Cano              | Ibarra
CL000007    | Gina     | Salas             | Reyes
CL000008    | Hugo     | Cruz              | Delgado
CL000009    | Ivana    | Ortega            | León
CL000010    | Jorge    | Pineda            | Ruiz
CL000011    | Laura    | Méndez            | Castillo
CL000012    | Miguel   | Ríos              | Guerrero
CL000013    | Natalia  | Vargas            | Silva
CL000014    | Óscar    | Delgado           | Peña
CL000015    | Patricia | Castro            | Ríos
CL000016    | Sofía    | Martínez          | López
(15 filas)
```

```
[spa=> select * from contratan;
num_cliente | id_paquete | fecha_c | metodo_pago
-----+-----+-----+-----
CL000002    | PK000002   | 2025-05-02 | Efectivo
CL000003    | PK000003   | 2025-05-03 | Transferencia
CL000004    | PK000004   | 2025-05-04 | TarjetaDébito
CL000005    | PK000005   | 2025-05-05 | Efectivo
CL000006    | PK000006   | 2025-05-06 | Depósito
CL000007    | PK000007   | 2025-05-07 | Transferencia
CL000008    | PK000008   | 2025-05-08 | TarjetaDébito
CL000009    | PK000009   | 2025-05-09 | Efectivo
CL000010    | PK000010   | 2025-05-10 | Transferencia
CL000002    | PK000003   | 2025-05-16 | Efectivo
CL000003    | PK000003   | 2025-05-17 | Transferencia
CL000011    | PK000003   | 2025-05-18 | TarjetaDébito
CL000012    | PK000003   | 2025-05-19 | Efectivo
CL000004    | PK000001   | 2025-05-20 | Depósito
CL000005    | PK000001   | 2025-05-21 | Transferencia
CL000013    | PK000004   | 2025-05-22 | TarjetaCrédito
CL000014    | PK000004   | 2025-05-23 | Efectivo
CL000015    | PK000002   | 2025-05-24 | TarjetaDébito
(18 filas)
```

```
[spa=> CALL sp_borra('empleados', 'EM000001');
NOTICE: Registro: EM000001 eliminado de la tabla empleados
CALL
```



```
[spa=> SELECT*FROM REALIZAN;
id_servicio | num_empleado | fecha_r
-----+-----+-----
SV000001    | EM000002     | 2025-05-11
SV000002    | EM000005     | 2025-05-11
SV000003    | EM000007     | 2025-05-12
SV000004    | EM000007     | 2025-05-12
SV000005    | EM000010     | 2025-05-12
SV000006    | EM000008     | 2025-05-13
SV000009    | EM000002     | 2025-05-14
SV000010    | EM000010     | 2025-05-15
SV000006    | EM000006     | 2024-04-08
SV000006    | EM000008     | 2024-03-21
(10 filas)
```

```
[spa=> SELECT*FROM EMPLEADOS;
num_empleado | nombres_e | apellido_paterno_e | apellido_materno_e | categoria_e
-----+-----+-----+-----+-----
EM000002     | Luis      | Gómez              | Rico                | Masajista
EM000003     | Sofía     | Campos             | Luna                | Recepcionista
EM000004     | Ricardo   | Díaz               | Neri                | Gerente
EM000005     | Valeria   | Beltrán            | Iglesias            | Esteticista
EM000006     | Tomás     | Quintero           | Solís               | Masajista
EM000007     | Daniela   | Rivas              | Toledo              | Esteticista
EM000008     | Pablo     | Acosta             | Serna               | Terapeuta
EM000009     | Nora      | Flores             | Vite                | Recepcionista
EM000010     | Ángel     | Santos             | Cuevas              | Masajista
EM000011     | Laura     | Díaz               | Romero              | Esteticista
(10 filas)
```

```
[spa=> CALL sp_borra('paquetes', 'PK000001');
ERROR: No se puede eliminar. El registro tiene relaciones en otras tablas.
CONTEXTO: función PL/pgSQL sp_borra(character varying,character varying) en la línea 38 en RAISE
```

```
[spa=> CALL sp_borra('servicios', 'SV000001');
ERROR: No se puede eliminar. El registro tiene relaciones en otras tablas.
CONTEXTO: función PL/pgSQL sp_borra(character varying,character varying) en la línea 38 en RAISE
```

```
[spa=> CALL sp_borra('camas_solares', 'CS000001');
NOTICE: Registro: CS000001 eliminado de la tabla camas_solares
CALL
```



```
[spa=> SELECT * FROM CAMAS_SOLARES;
num_serie | num_focos
-----+-----
CS000002 |         28
CS000003 |         32
CS000004 |         36
CS000005 |         40
CS000006 |         24
CS000007 |         28
CS000008 |         32
CS000009 |         36
CS000010 |         40
CS000011 |         42
(10 filas)
```

```
[spa=> SELECT * FROM OFRECEN;
id_servicio | num_serie | id_instalacion
-----+-----+-----
SV000006 | CS000002 | IN000002
SV000006 | CS000003 | IN000003
SV000006 | CS000004 | IN000004
SV000006 | CS000005 | IN000005
SV000005 | CS000006 | IN000006
SV000005 | CS000007 | IN000007
SV000005 | CS000008 | IN000008
SV000005 | CS000009 | IN000009
SV000005 | CS000010 | IN000010
(9 filas)
```

```
[spa=> CALL sp_borra('instalaciones', 'IN000001');
NOTICE: Registro: IN000001 eliminado de la tabla instalaciones
CALL
```

```
[spa=> SELECT * FROM INSTALACIONES;
id_instalacion | num_camas
-----+-----
IN000002 |         10
IN000003 |          6
IN000004 |         12
IN000005 |          9
IN000006 |          7
IN000007 |         11
IN000008 |         10
IN000009 |          8
IN000010 |         14
IN000011 |         15
(10 filas)
```

```

spa=> SELECT * FROM OFRECEN;
id_servicio | num_serie | id_instalacion
-----+-----+-----
SV000006    | CS000002  | IN000002
SV000006    | CS000003  | IN000003
SV000006    | CS000004  | IN000004
SV000006    | CS000005  | IN000005
SV000005    | CS000006  | IN000006
SV000005    | CS000007  | IN000007
SV000005    | CS000008  | IN000008
SV000005    | CS000009  | IN000009
SV000005    | CS000010  | IN000010
(9 filas)

```

Comandos necesarios para el control de la integridad de la información.

Consideramos que incluimos los comandos necesarios para el control de la integridad de la información, con los constraints en el script de 'integridad_spa_f3.sql', y las validaciones dentro de los procedimientos almacenados de sp_inserta que garantizan que no se inserten datos no válidos a la base de datos. Además, incluimos índices únicos como el de: idx_empleados_nombre_unique o idx_clientes_nombre_unique.

También, adjuntamos un *commit* en el *script* de 'registros_spa_f3.sql' para garantizar la integridad de las transacciones (insertar registros porque son 'muchos').

```
DO $$
BEGIN

-- Clientes
INSERT INTO Clientes (num_cliente, nombres_c, apellido_paterno_c, apellido_materno_c)
VALUES
('CL000001', 'Ana María', 'López', 'García'),
('CL000002', 'Bruno', 'Hernández', 'Mora')
```

```
Registros_spa_f3.sql — Edited

-- Realizan
INSERT INTO Realizan (id_servicio, num_empleado, fecha_r) VALUES
('SV000001', 'EM000002', '2025-05-11'),
('SV000002', 'EM000005', '2025-05-11'),
('SV000003', 'EM000007', '2025-05-12'),
('SV000004', 'EM000007', '2025-05-12'),
('SV000005', 'EM000010', '2025-05-12'),
('SV000006', 'EM000008', '2025-05-13'),
('SV000007', 'EM000001', '2025-05-13'),
('SV000008', 'EM000001', '2025-05-14'),
('SV000009', 'EM000002', '2025-05-14'),
('SV000010', 'EM000010', '2025-05-15'),
('SV000006', 'EM000006', '2024-04-08'),
('SV000006', 'EM000008', '2024-03-21');

-- Todas las inserciones anteriores son exitosas
COMMIT;

-- Manejo de errores
EXCEPTION
WHEN OTHERS THEN
-- Si ocurre cualquier error, deshacemos todas las inserciones de la transaccion
ROLLBACK;
RAISE NOTICE 'Hubo un error al insertar los registros. No se realizo ninguna
transacción.';
END $$;
```

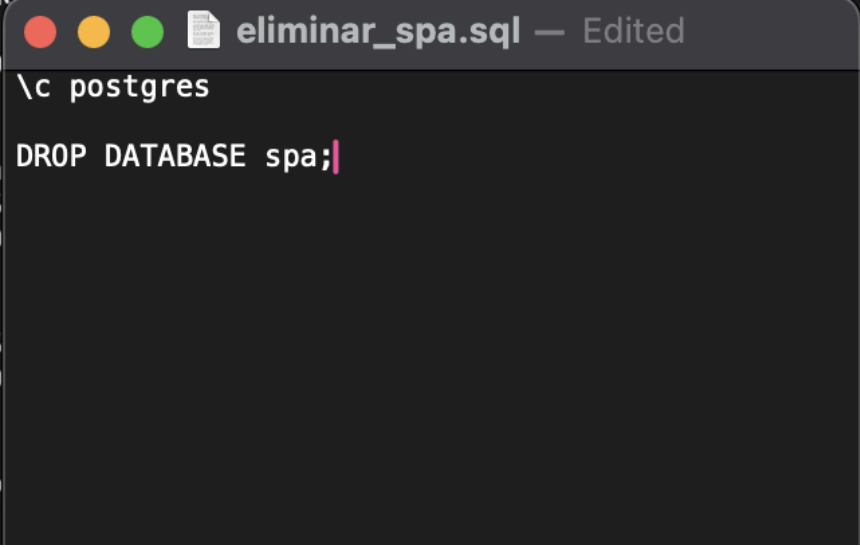
Como ya se habían insertado anteriormente, si los volvemos a insertar nos debe manejar el error de que ya existen en la base de datos:

```
milenafer — alumno06@master:~ — ssh -p2206 alumno06@132.248.51.117 — 107x31

INSERT INTO Clientes (num_cliente, nombres_c, apellido_paterno_c, apellido_materno_c) VALUES
('CL000001','Ana Maria','López','García'),
('CL000002','Bruno','Hernández','Mora'),
('CL000003','Carla','Ramírez','Soto'),
('CL000004','Diego','Núñez','Rojas'),
('CL000005','Elena','Vega','Paz'),
('CL000006','Fabio','Cano','Ibarra'),
('CL000007','Gina','Salas','Reyes'),
('CL000008','Hugo','Cruz','Delgado'),
('CL000009','Ivana','Ortega','León'),
('CL000010','Jorge','Pineda','Ruiz'),
('CL000011','Laura','Méndez','Castillo'),
('CL000012','Miguel','Ríos','Guerrero'),
('CL000013','Natalia','Vargas','Silva'),
('CL000014','Óscar','Delgado','Peña'),
('CL000015','Patricia','Castro','Ríos');

-- Paquetes
INSERT INTO Paquetes (id_paquete, nombre_paquete) VALUES
('PK000001','Básico'),
('PK000002','Relax'),
('PK000003','Premium'),
('PK000004','Novias'),
('PK000005','Detox'),
('PK000006','Hidratación'),
('PK000007','Express'),
('PK000008','Gold');
END $$; RAISE NOTICE 'Hubo un error al insertar los registros. No se realizo ninguna transacción.';egoria_e
NOTICE: Hubo un error al insertar los registros. No se realizo ninguna transacción.
DO
spa=>
```


Código para borrar la base de datos.

A screenshot of a code editor window. The title bar at the top shows three colored circles (red, yellow, green) and a document icon, followed by the text 'eliminar_spa.sql — Edited'. The main area of the editor is dark and contains the text '\c postgres' on the first line and 'DROP DATABASE spa;' on the second line. The text is in a light color, likely white or light gray, with some syntax highlighting. The cursor is positioned at the end of the second line.

NO lo ejecutamos para que SPA aún aparezca en la base de datos de la clase.